

Winard

a window,
controlled by a microcontroller,
programmed by arduino.

By
Wil Straver

Version 0.1

July 2025

Table of Contents

1 Introduction.....	3
2 Windows.....	7
2.1 The primary Window.....	7
2.2 Toplevel window.....	7
3 Widgets.....	8
3.1 containers.....	8
3.1.1 Frame.....	8
3.1.2 Labeled frame.....	8
3.1.3 Notebook.....	9
3.2 Buttons.....	10
3.2.1 button, radiobutton and checkbutton.....	10
3.2.2 menubuttons.....	11
3.2.3 menubar.....	11
3.2.4 separator.....	12
3.3 Entry widgets.....	13
3.3.1 Slider or scale.....	13
3.3.2 Spinbox.....	14
3.3.3 Entry.....	15
3.4 Canvas.....	16
3.4.1 Grid lines.....	17
3.4.2 Line.....	17
4 Themes and styles.....	19
4.1 Toplevel.....	20
4.2 Text style.....	21
4.3 Buttons style.....	23
4.3.1 button, radiobutton and checkbutton.....	23
4.3.2 Style map.....	24
4.3.3 Button size.....	25
4.4 Frame.....	26
4.5 Notebook.....	27
4.6 Menubutton.....	28
4.7 Menubar.....	29
4.8 Slider or scale.....	30
4.9 Spinbox.....	31
4.10 Entry.....	32
5 Interaction PC-window and microcontroller.....	33
5.1 Display an MC digital input signal on Winard.....	35
5.2 Draw an MC analog input signal on Winard.....	36
5.3 Switch LEDs connected to the MC on or off via Winard.....	37
5.4 Led brightness with slider.....	38
5.5 Draw functions.....	39
6 Winard.h.....	41

1 Introduction

In the following: MC means microcontroller

Winard is a program written in Python. It starts a window whose dimensions are determined by a connected MC. The window's contents are also populated by commands from the MC. These commands can include: place a button, radio button, canvas, notebook, frame, entry, etc. There are passive elements such as frames, lines, etc., but also active elements such as buttons, sliders, etc. When an active element is activated, a code is sent from the window (PC) to the MC. The MC processes this code and can send a response back to the PC and/or control an element connected to the MC, such as an LED or motor. A digital or analog signal connected to the MC can also be processed and forwarded to Winard on the PC. An analog signal converted into digital information by the MC's ADC can then be displayed on a canvas. Rectangles, ellipses, lines, etc., can also be drawn on the canvas. This makes the whole thing work similarly to a touchscreen connected directly to the MC.

Winard was developed on LinuxMint and hasn't been tested on other operating systems. However, because Python and tkinter are used, I expect it will work with other OSes, with or without some modifications.

Winard uses the following import modules

```
from tkinter import *
import tkinter as tk
from tkinter.ttk import *
import tkinter.font as tkFont
from functools import partial
from tkinter import messagebox
#from ttkthemes import ThemedTk
import time
import serial
import sys
```

Sequence of operations:

Create the desired program in the Arduino IDE,
compile and upload it.

Keep the microcontroller connected to the PC.

Launch Winard from a terminal: `python3 winard.py /dev/ttyUSB0`

The last argument is the USB port, which varies by OS and/or USB port number..

All examples used the Arduino Nano, several were also tested with the stm32f103c8.

You might wonder why the style programming method differs for one widget from another. Winard uses both tkinter and ttkinter. Ttkinter is a more modern version of tkinter, but not everything can be programmed in ttkinter, making Winard a mix of tkinter and ttkinter. For more information, see the python, tkinter, and ttkinter documentation.

Next page gives an example of how to program a window with content.

code microcontroller

1830 bytes of program storage space

```
void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

void build_up()
{ Serial.println("105,1,");
  Serial.println("2,Arduino Display 1");
  Serial.println("41,0,320,240,");
  Serial.println("2,Build up with Tkinter,");
  Serial.println("24,3,41,0,100,35,");
  Serial.println("2,button");
  Serial.println("25,6,41,0,120,80,");
  Serial.println("2,radiobutton1");
  Serial.println("26,7,41,0,110,120,");
  Serial.println("2,radiobutton2");
  Serial.println("26,8,41,0,110,140,");
  Serial.println("2,checkboxbutton");
  Serial.println("27,9,41,0,110,160,");
  Serial.println("104,1,");
}
```

code, more readable, same result

3448 bytes of program storage space

```
# define TEXT      2
# define LABEL    24
# define BUTTON   25
# define RBUTTON  26
# define CBUTTON  27
# define ROOT     41
# define PRINTOBJECTS 104
# define PRINTBUILD 105

char print_str[64];

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

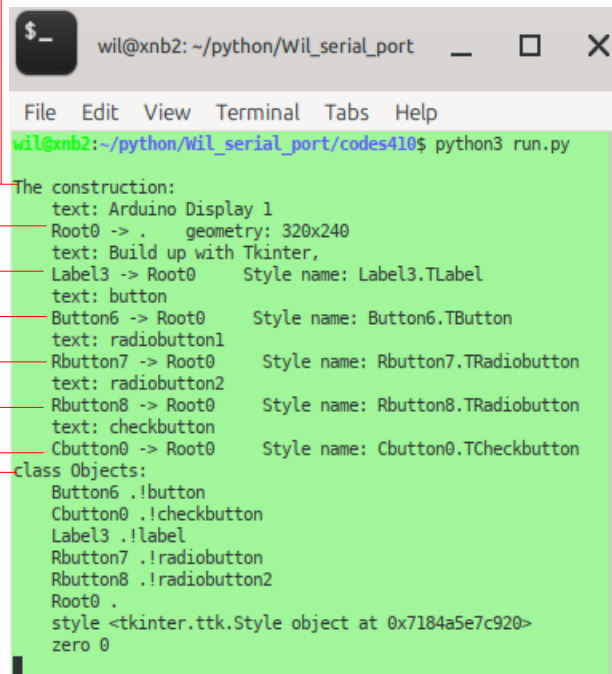
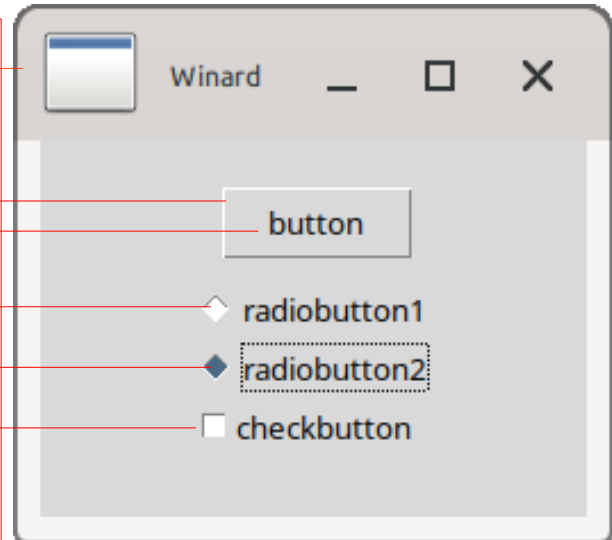
void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Arduino Display");
  tx_04(ROOT,0,320,240);
  tx_str(TEXT,"Build up with Tkinter");
  tx_06(LABEL,3,ROOT,0,100,35);
  tx_str(TEXT,"button");
  tx_06(BUTTON,6,ROOT,0,120,80);
  tx_str(TEXT,"radiobutton1");
  tx_06(RBUTTON,7,ROOT,0,110,120);
  tx_str(TEXT,"radiobutton2");
  tx_06(RBUTTON,8,ROOT,0,110,145);
  tx_str(TEXT,"checkboxbutton");
  tx_06(CBUTTON,9,ROOT,0,110,170);
  tx_02(PRINTOBJECTS,1);
}

void tx_02(int a,int b)
{ sprintf(print_str,"%d,%d,",a,b);
  Serial.println(print_str);
}

void tx_04(int a,int b,int c,int d)
{ sprintf(print_str,"%d,%d,%d,%d,",a,b,c,d);
  Serial.println(print_str);
}

void tx_06(int a,int b,int c,int d,int e,int f)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,",a,b,c,d,e,f);
  Serial.println(print_str);
}

void tx_str(int a,const char *str)
{ sprintf(print_str,"%d,%s",a,str);
  Serial.println(print_str);
}
```



Short explanation of coding, the yellow rectangle:

```
Serial.println(2,button);
Serial.println(25,6,41,0,120,80,);
```

token of a text string
text string
y position
x position
token destination serie number
token destination, in this case the root window
token serie number of the widget
token number, in this case the button widget.

In tkinter language:

The first token is the child widget and the second token, the destination, is the parent widget.

When macros are used the code becomes more readable.

The code, in the light blue rectangle:

```
# define TEXT          2
# define BUTTON       25
etc.

tx_str(TEXT,"button");
tx_06(BUTTON,6,ROOT,0,120,80);
etc

void tx_str(int a,const char *str)
{ sprintf(print_str,"%d,%s",a,str);
  Serial.println(print_str);
}

void tx_06(int a,int b,int c,int d,int e,int f)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d",a,b,c,d,e,f);
  Serial.println(print_str);
}
etc.
```

A collection of macros and functions are added if we include the header file <winard.h>.

From here on, all examples will use the include file <winard.h>.

The transferred code can be displayed by Arduino SerialMonitor when Winard is not running.

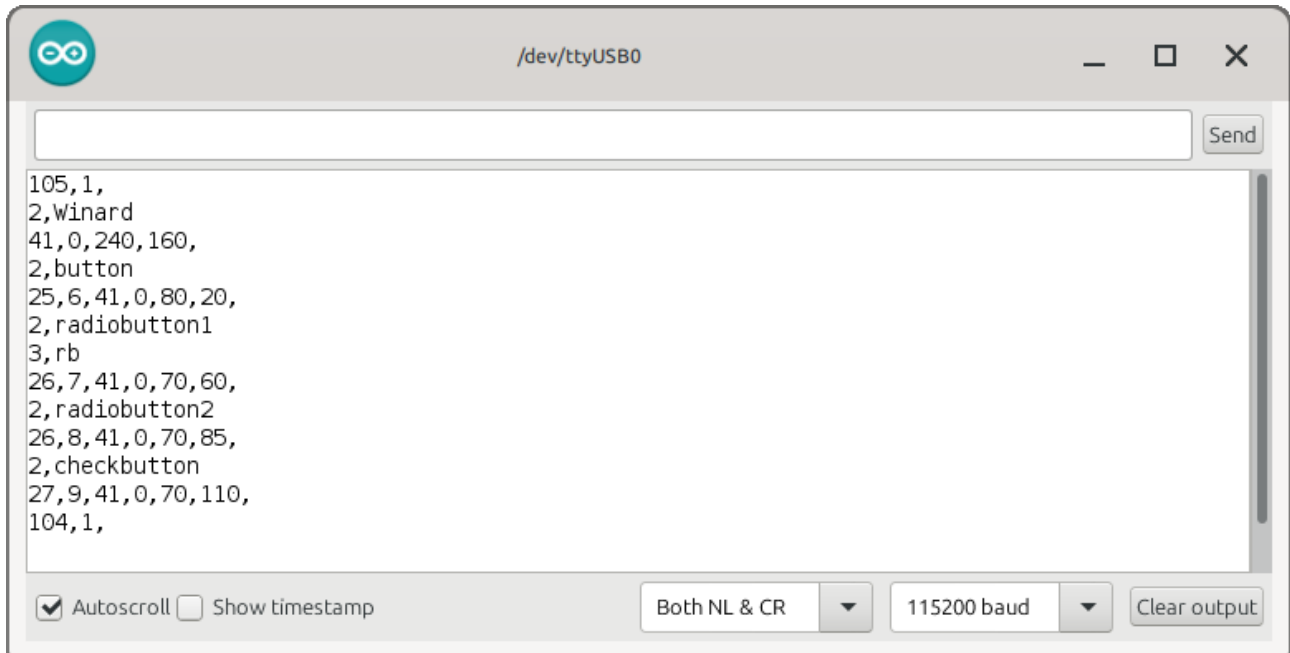
In the Arduino IDE, select:

Tools → SerialMonitor

then:

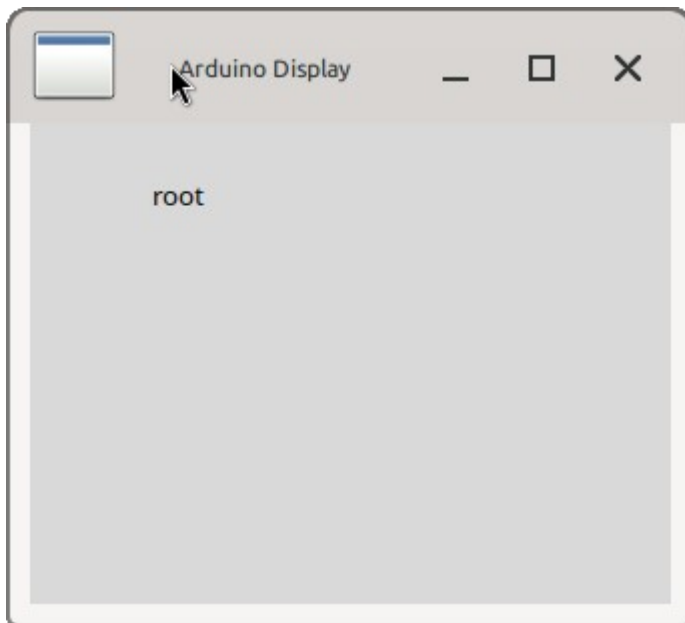
Reset microcontroller

Result:



2 Windows

2.1 The primary Window



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {;}

void build_up()
{ tx_02(PRINTBUILD,1);

  tx_str(TEXT,"Arduino Display");
  tx_04(ROOT,0,320,240);

  tx_str(TEXT,"root");
  tx_06(LABEL,0,ROOT,0,60,25);

  tx_02(PRINTOBJECTS,1);
}
```

2.2 Toplevel window

Child of the primary window

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {;}

void build_up()
{ tx_02(PRINTBUILD,1);

  tx_str(TEXT,"Display 0");
  tx_04(ROOT,0,320,240);

  tx_str(TEXT,"Display 1");
  tx_04(TOPLEVEL,1,320,240);

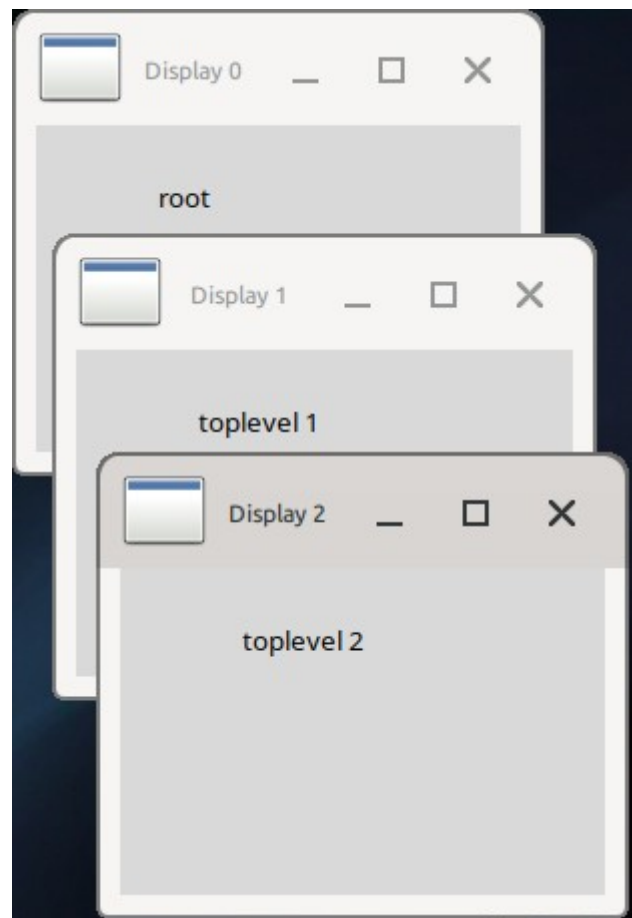
  tx_str(TEXT,"Display 2");
  tx_04(TOPLEVEL,2,320,240);

  tx_str(TEXT,"root");
  tx_06(LABEL,0,ROOT,0,60,25);

  tx_str(TEXT,"toplevel 1");
  tx_06(LABEL,1,TOPLEVEL,1,60,25);

  tx_str(TEXT,"toplevel 2");
  tx_06(LABEL,2,TOPLEVEL,2,60,25);

  tx_02(PRINTOBJECTS,1);
}
```



3 Widgets

3.1 containers

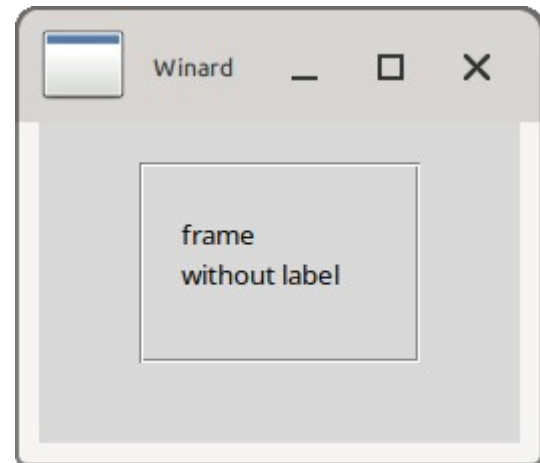
3.1.1 Frame

```
#include <winard.h>

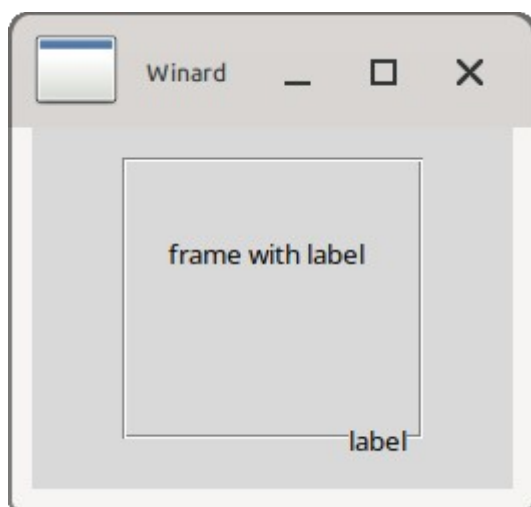
void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);
  tx_08(FRAME,1,ROOT,0,50,20,140,100);
  tx_str(TEXT,"frame");
  tx_06(LABEL,4,FRAME,1,20,25);
  tx_str(TEXT,"without label");
  tx_06(LABEL,5,FRAME,1,20,45);
  tx_02(RELIEF,1);
  tx_05(STYLE,0,FRAME,1,RELIEF);
  tx_02(BORDERWIDTH,2);
  tx_05(STYLE,0,FRAME,1,BORDERWIDTH);
  tx_02(PRINTOBJECTS,1);
}
```



3.1.2 Labeled frame



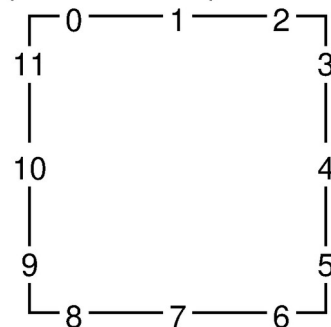
```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,180);
  tx_str(TEXT,"label");
  tx_02(POSITION,6);
  tx_08(LABELFRAME,1,ROOT,0,45,15,150,150);
  tx_str(TEXT,"frame with label");
  tx_06(LABEL,5,LABELFRAME,1,20,35);
  tx_02(PRINTOBJECTS,1);
}
```

possible label positions



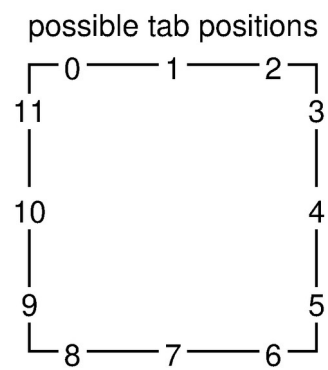
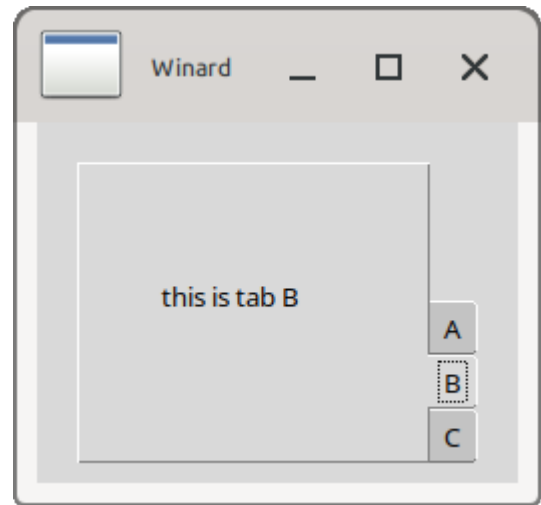
3.1.3 Notebook

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

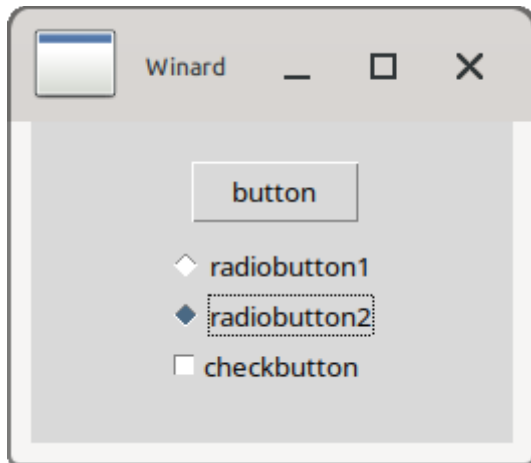
void loop() {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,180);
  tx_str(TEXT,"A,B,C");
  tx_02(POSITION,5);
  tx_08(NOTEBOOK,1,ROOT,0,20,20,200,150);
  tx_str(TEXT,"this is tab A");
  tx_06(LABEL,4,TAB,0,40,35);
  tx_str(TEXT,"this is tab B");
  tx_06(LABEL,5,TAB,1,40,55);
  tx_str(TEXT,"this is tab C");
  tx_06(LABEL,6,TAB,2,40,75);
  tx_02(PRINTOBJECTS,1);
}
```



3.2 Buttons

3.2.1 button, radiobutton and checkbutton



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);
  tx_str(TEXT,"button");
  tx_06(BUTTON,6,ROOT,0,80,20);
  tx_str(TEXT,"radiobutton1");
  tx_str(TAGS,"rb");
  tx_06(RBUTTON,7,ROOT,0,70,60);
  tx_str(TEXT,"radiobutton2");
  tx_06(RBUTTON,8,ROOT,0,70,85);
  tx_str(TEXT,"checkbutton");
  tx_06(CBUTTON,9,ROOT,0,70,110);
  tx_02(PRINTOBJECTS,1);
}
```

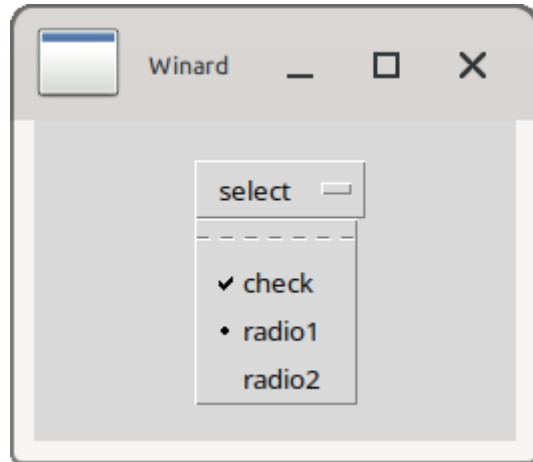
3.2.2 menubuttons

```
#include <winard.h>

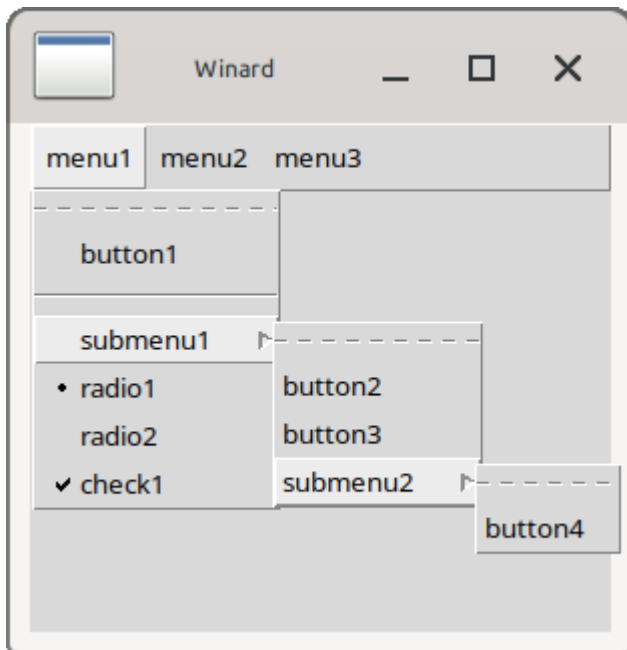
void setup( )
{ Serial.begin(115200);
  the_build();
}

void loop() {}

void the_build()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);
  tx_str(TEXT,"select");
  tx_06(MBUTTON,2,ROOT,0,80,20);
  tx_str(TEXT,"check");
  tx_04(MCBUTTON,3,MBUTTON,2);
  tx_str(TAGS,"rb");
  tx_str(TEXT,"radio1");
  tx_04(MRBUTTON,5,MBUTTON,2);
  tx_str(TEXT,"radio2");
  tx_04(MRBUTTON,6,MBUTTON,2);
  tx_02(PRINTOBJECTS,1);
}
```



3.2.3 menubar



```
#include <winard.h>
void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,290,220);
  tx_02(TEAR,1);
  tx_str(TEXT,"menu1,menu2,menu3");
  tx_04(MENUBAR,0,ROOT,0);
  tx_str(TEXT,"button1");
  tx_04(BMCOMMAND,1,BARMENU,0);
  tx_04(BMSEPARATOR,2,BARMENU,0);
  tx_str(TEXT,"submenu1");
  tx_04(SUBMENU,0,BARMENU,0);
  tx_str(TEXT,"button2");
  tx_04(BMCOMMAND,2,SUBMENU,0);
  tx_str(TEXT,"button3");
  tx_04(BMCOMMAND,3,SUBMENU,0);
  tx_str(TEXT,"submenu2");
  tx_04(SUBMENU,2,SUBMENU,0);
  tx_str(TEXT,"button4");
  tx_04(BMCOMMAND,4,SUBMENU,2);
  tx_str(TAGS,"rb2");
  //TAGS, connects the radio buttons
  // with this label
  tx_str(TEXT,"radio1");
  tx_04(BMRADIO,1,BARMENU,0);
  tx_str(TEXT,"radio2");
  tx_04(BMRADIO,2,BARMENU,0);
  tx_str(TEXT,"check1");
  tx_04(BMCHECK,1,BARMENU,0);
  tx_str(TEXT,"button5");
  tx_04(BMCOMMAND,5,BARMENU,1);
  tx_str(TEXT,"button6");
  tx_04(BMCOMMAND,6,BARMENU,1);
  tx_04(BMSEPARATOR,0,BARMENU,1);
  tx_str(TEXT,"button7");
  tx_04(BMCOMMAND,7,BARMENU,1);
  tx_02(PRINTOBJECTS,1);
}
```

3.2.4 separator

With separator you can draw a vertical or horizontal line.

```
#include <winard.h>

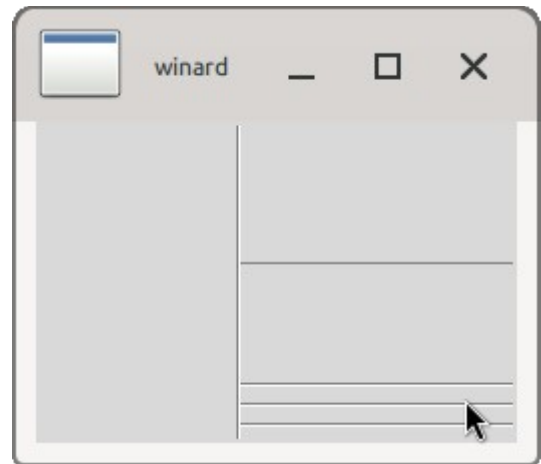
void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

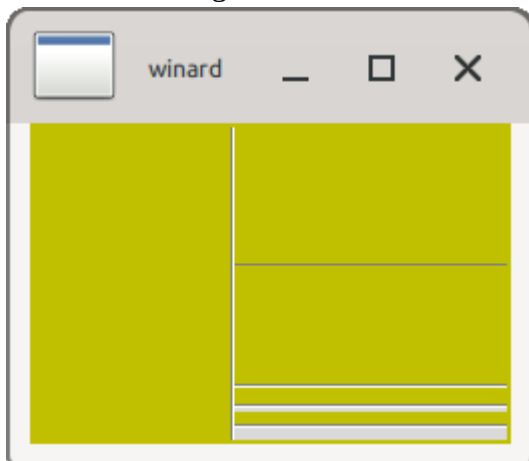
void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"winard");
  tx_04(ROOT,0,240,160);

  tx_08(SEPARATOR,1,ROOT,0,100,70,138,1);
  tx_08(SEPARATOR,1,ROOT,0,100,130,138,2);
  tx_08(SEPARATOR,1,ROOT,0,100,140,138,4);
  tx_08(SEPARATOR,1,ROOT,0,100,150,138,8);
  tx_08(SEPARATOR,2,ROOT,0,100,2,2,156);

  tx_02(PRINTOBJECTS,1);
}
```



To see better the line width,
use an other background color:



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"winard");
  tx_04(ROOT,0,240,160);

  tx_04(CBG,YELLOW);
  tx_05(STYLE,0,ROOT,0,CBG);

  tx_08(SEPARATOR,1,ROOT,0,100,70,138,1);
  tx_08(SEPARATOR,1,ROOT,0,100,130,138,2);
  tx_08(SEPARATOR,1,ROOT,0,100,140,138,4);
  tx_08(SEPARATOR,1,ROOT,0,100,150,138,8);
  tx_08(SEPARATOR,2,ROOT,0,100,2,2,156);

  tx_02(PRINTOBJECTS,1);
}
```

Horizontal line:

```
tx_10(SEPARATOR,1,ROOT,0,100,70,138,1);
```

Diagram illustrating the parameters of the `tx_10` function for a horizontal line:

- separator number
- destination
- xstart
- y start
- length
- width

Vertical line:

```
tx_10(SEPARATOR,2,ROOT,0,100,2,2,156);
```

Diagram illustrating the parameters of the `tx_10` function for a vertical line:

- length
- width

3.3 Entry widgets

3.3.1 Slider or scale

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

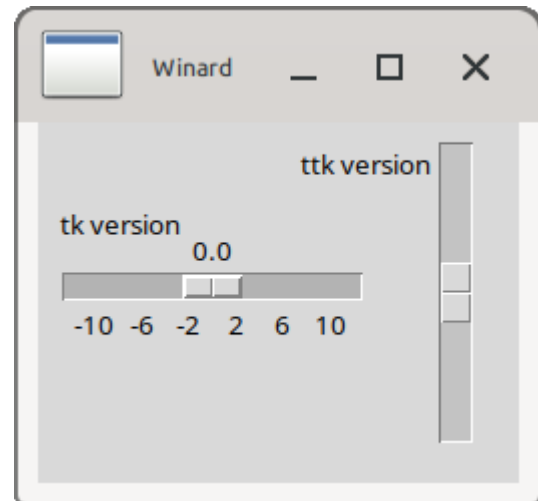
void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,180);

  tx_str(TEXT,"0 1 4.0 -10.0 10.0 2.0 1");
  tx_08(SCALE,18,ROOT,0,10,50,150,12);
  tx_str(TEXT,"tk version");
  tx_06(LABEL,1,ROOT,0,10,40);

  tx_str(TEXT,"1 1 2.0 20.0 -20.0 2.0 2");
  tx_08(SCALE,19,ROOT,0,200,10,0,150);
  tx_str(TEXT,"ttk version");
  tx_06(LABEL,1,ROOT,0,130,10);

  tx_02(PRINTOBJECTS,1);
}
```



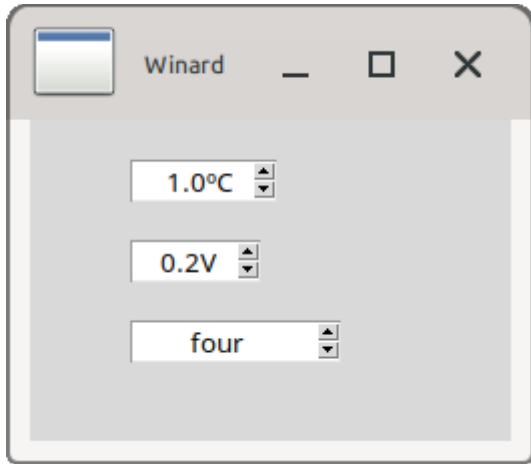
```
tx_str(TEXT,"0 1 4.0 -10.0 10.0 2.0 1");
```

resolution 1=10⁻¹ 2=10⁻² etc.
set
to
from
interval
showvalue
type 0=tk 1=ttk

```
tx_08(SCALE,18,ROOT,0,10,50,150,12);
```

Y2,(if Y2 < X2) Y2=width //vertical
else: Y2=length
X2,(if X2 < Y2) X2=width //horizontal
else: X2=length
Y1 start coord.
X1 start coord.

3.3.2 Spinbox



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Arduino Display");
  tx_04(ROOT,0,240,160);

  tx_str(SPINBOXSET,"0 2 6 1.0°C");
  tx_str(TEXT,"0.0 10.0 0.1 %.1f°C ");
  tx_06(SPINBOX,1,ROOT,0,50,20);

  // remember, next line not longer then the
  // buffer length, including TEXT, spaces
  // and the two newline/feedback characters

  tx_str(TEXT,"-0.2V -0.1V 0.0V 0.1V 0.2V
              0.5V 1.0V");
  tx_str(SPINBOXSET,"1 2 5 0.2V");
  tx_06(SPINBOX,2,ROOT,0,50,60);

  tx_str(SPINBOXSET,"1 1 10 four");
  tx_str(TEXT,"one two three four five");
  tx_06(SPINBOX,3,ROOT,0,50,100);
}
```

```
tx_str(SPINBOXSET,"0 2 6 1.0°C");
```

0 display type
 use TEXT:
 tx_str(TEXT,"0.0 10.0 0.1 %.1f°C ");
 format
 increment
 to
 from

1 display type
 use TEXT:
 tx_str(TEXT,"0.2 0.4 0.6 0.8 1.0 1.2")

3.3.3 Entry

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

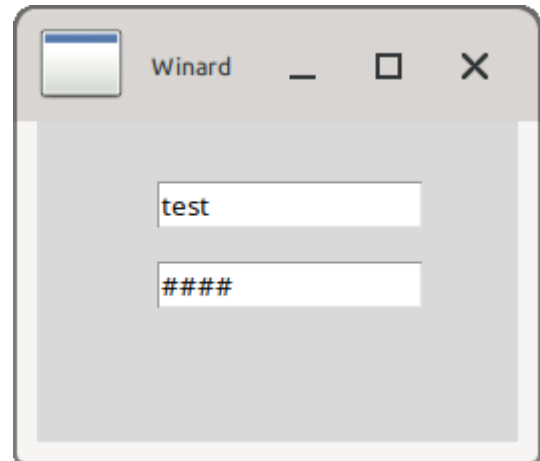
void loop() {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

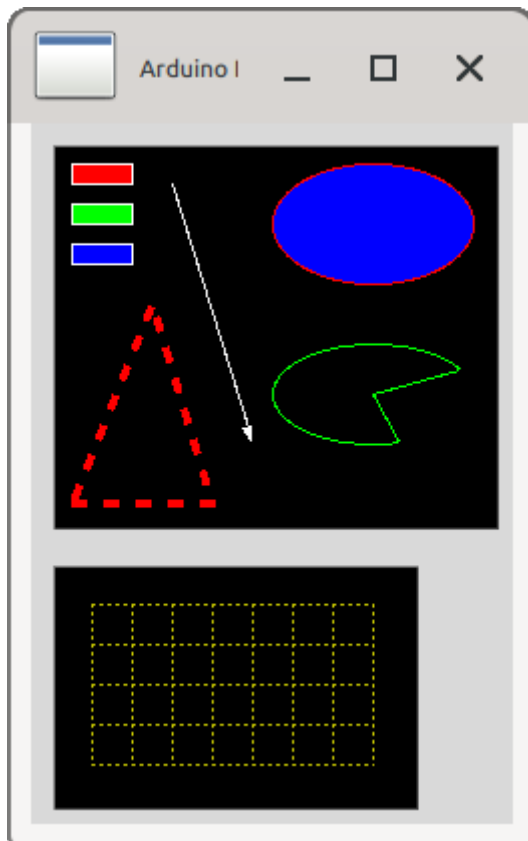
  tx_02(WIDTH,16);
  tx_06(ENTRY,3,ROOT,0,60,30);

  tx_str(SHOW,"#");
  tx_06(ENTRY,14,ROOT,0,60,70);

  tx_02(PRINTOBJECTS,1);
}
```



3.4 Canvas



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);      //(115200);
  build_up();
}

void loop()
{ delay(500) ;
  tx_02(CANVASSELECT,0);
  tx_str(TAGSDEL,"rect");
  delay(500) ;
  rectangles();
}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Arduino Display 0");
  tx_04(ROOT,0,240,350);

  tx_04(CFG,255,255,255);
  tx_04(CBG,0,0,0);
  tx_08(CANVAS,0,ROOT,0,10,10,220,190);

  rectangles();
  tx_str(TAGS,"line");
  tx_02(LINEWIDTH,1);
  tx_02(ARROW,2);
  tx_04(ARROW,14,20,8);
  tx_08(LINE,5,CANVAS,0,60,20,100,150);

  tx_04(DASH,1,8,8);
  tx_04(CFG,255,0,0);
  tx_02(ARROW,0);
  tx_02(LINEWIDTH,4);
  tx_12(LINE,6,CANVAS,0,10,180,80,180,50,80,10,180);
  tx_04(DASH,0,8,8);
  tx_02(LINEWIDTH,1);
  tx_04(CBG,0,0,255);
  tx_08(OVAL,1,CANVAS,0,110,10,210,70);
  // the oval is drawn in a rectangle X1,Y1  X2,Y2
  // draw a circle: X2-X1=Y2-Y1
  tx_04(CBG,0,0,0);
  tx_04(CFG,0,255,0);
  tx_04(ARCShape,30,255,1);
  //          | | |_0=arc 1=pieslice 2=chord
  //          | | |___extend corner
  //          | | |___start corner
  tx_08(ARC,1,CANVAS,0,110,100,210,150);
  // the underground shape of the arc is just like the
  oval

  tx_04(CBG,0,0,0);
  tx_08(CANVAS,1,ROOT,0,10,220,180,120);
  tx_05(NLINES,5,8,20,20);
  tx_04(CFG,200,200,0);
  tx_02(LINEWIDTH,1);
  tx_04(DASH,1,2,2);
  tx_08(VLINES,3,CANVAS,1,20,20,20,100);
  tx_08(HLINES,3,CANVAS,1,20,20,161,20);
  tx_04(DASH,0,2,2);
  // rectangles();

  tx_02(PRINTOBJECTS,1);
  tx_02(PRINTBUILD,0);
}

void rectangles()
{ tx_str(TAGS,"rect");
  tx_04(CFG,255,255,255);
  tx_04(CBG,255,0,0);
  tx_08(RECT,5,CANVAS,0,10,10,40,20);
  tx_04(CBG,0,255,0);
  tx_08(RECT,6,CANVAS,0,10,30,40,40);
  tx_04(CBG,0,0,255);
  tx_08(RECT,7,CANVAS,0,10,50,40,60);
}
```


3.4.1 Grid lines

Drawing a grid on canvas is common. A functionality has been added to Winard to plot such a grid with minimal code.

First:

```
tx_05(NLINES,nX,nY,disX,disY)
```

line distances in Y direction
line distances in X direction
number of vertical lines
number of horizontal lines

Draw the lines:

```
tx_08(VLINES,n,CANVAS,m,X1,Y1,X2,Y2)
```

```
tx_08(HLINES,n,CANVAS,m,X1,Y1,X2,Y2)
```

3.4.2 Line

To draw a line, use:

```
tx_xx(LINE,n,CANVAS,m,X1,Y1,X2,Y2,X3,.....etc.).
```

xx (2 3 4 ... 12) is the number of arguments.

The data to be transferred is stored in a buffer. The size of this buffer is limited to, for example, 64 bytes. This includes all characters, such as the coding of LINE and CANVAS plus their numbers, the coordinates, commas, and the end-of-line characters. This limits the number of vertices.

In order to create a line with many coordinates, some functions have been added to Winard. Winard includes a buffer array, the length of which is automatically adjusted to the amount of data. Starting and initializing this buffer:

```
tx_xx(COORD,a,b,c,d,.....etc)
```

a,b,c,d,.....etc is used instead of X1,Y1,X2,Y2,.... The reason will become clear later. Note the maximum length of the microcontroller's buffer. Adding data to the Winard buffer:

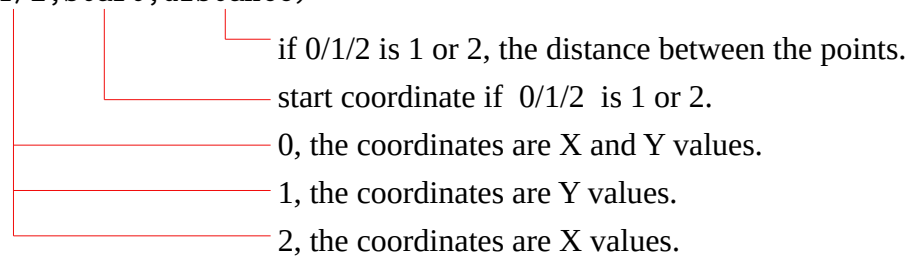
```
tx_xx(COORDAPPEND,a,b,c,d,.....etc)
```

Replace some of the Winard buffer data:

```
tx_xx(COORDREPLACE,replace from,a,b,c,d,.....etc)
```

The following command specifies how the data from the Winard buffer will be used.

`tx_04(SETLINE,0/1/2,start,distance)`



Draw the line:

`tx_04(NEWLINE,n,CANVAS,m)` where n is the line number and m is the canvas number.

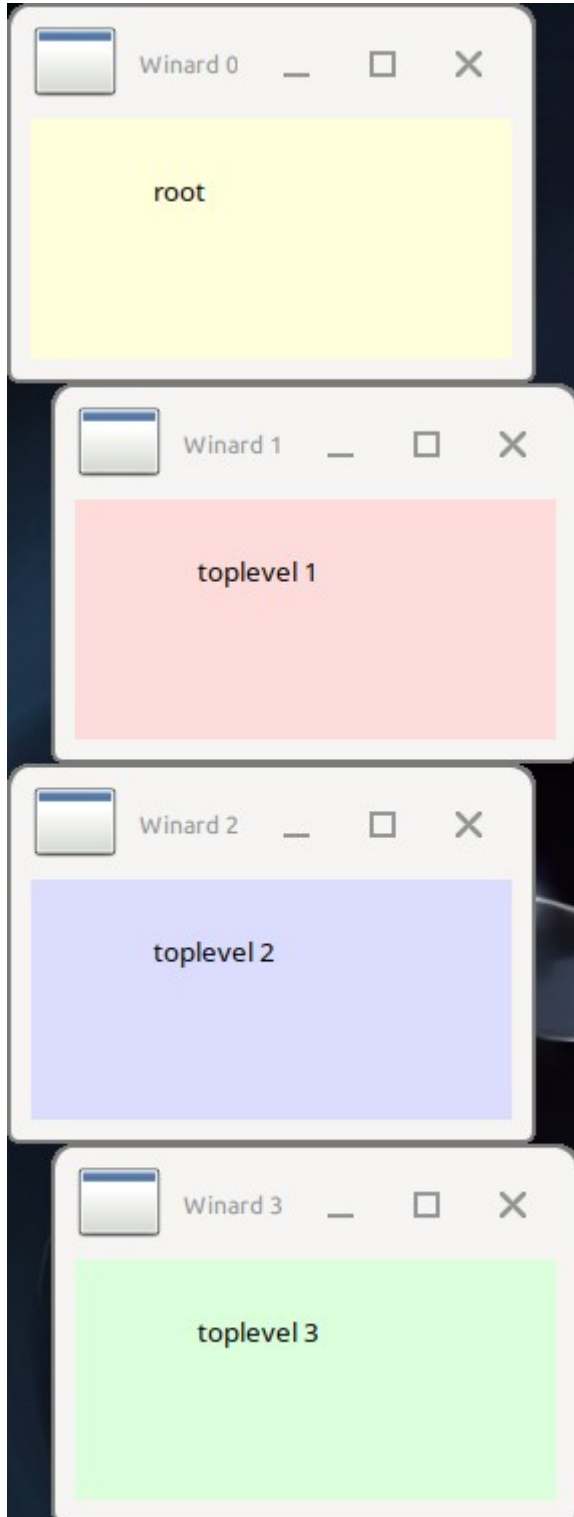
4 Themes and styles

Themes is a collection of styles.

In a theme, all widgets have a default style. It is possible to change the style of a particular widget, for example a button, individually or all of them.

When the style of a widget is changed and then the theme is changed, the style of the new theme is visible, not the changed style.

4.1 Toplevel



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

void build_up()
{ tx_02(PRINTBUILD,1);
  int x=240, y=120;

  tx_str(TEXT,"Winard 0");
  tx_04(ROOT,0,x,y);

  tx_str(TEXT,"Winard 1");
  tx_04(TOPLEVEL,1,x,y);

  tx_str(TEXT,"Winard 2");
  tx_04(TOPLEVEL,2,x,y);

  tx_str(TEXT,"Winard 3");
  tx_04(TOPLEVEL,3,x,y);

  tx_str(TEXT,"root");
  tx_06(LABEL,0,ROOT,0,60,25);

  tx_str(TEXT,"toplevel 1");
  tx_06(LABEL,1,TOPLEVEL,1,60,25);

  tx_str(TEXT,"toplevel 2");
  tx_06(LABEL,2,TOPLEVEL,2,60,25);

  tx_str(TEXT,"toplevel 3");
  tx_06(LABEL,3,TOPLEVEL,3,60,25);

  tx_04(CBG,255,255,220);
  tx_05(STYLE,0,ROOT,0,CBG);
  tx_05(STYLE,0,LABEL,0,CBG);

  tx_04(CBG,255,220,220);
  tx_05(STYLE,0,TOPLEVEL,1,CBG);
  tx_05(STYLE,0,LABEL,1,CBG);

  tx_04(CBG,220,220,255);
  tx_05(STYLE,0,TOPLEVEL,2,CBG);
  tx_05(STYLE,0,LABEL,2,CBG);

  tx_04(CBG,220,255,220);
  tx_05(STYLE,0,TOPLEVEL,3,CBG);
  tx_05(STYLE,0,LABEL,3,CBG);

  tx_02(PRINTOBJECTS,1);
}
```

4.2 Text style

```
tx_10(FONT,0,BUTTON,6,2,16,0,1,0,0);
```

strike
 underlined
 italic
 bold
 size
 fonttype 0 Helvetica 1 Times 2 Arial 3 Verdana
 destination
 nothing here, but used in the case of labelframe and notebook.
 font style

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

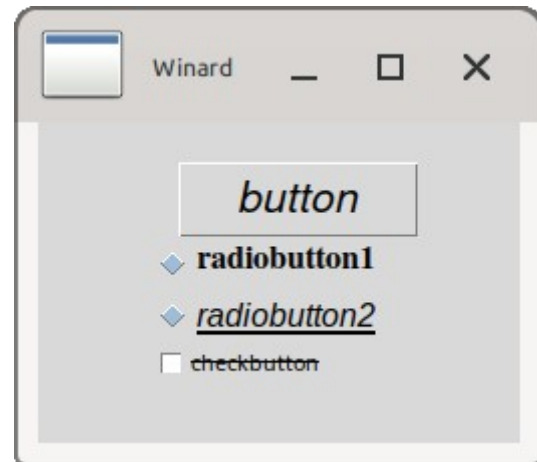
  tx_str(TEXT,"button");
  tx_06(BUTTON,6,ROOT,0,70,20);
  tx_10(FONT,0,BUTTON,6,0,16,0,1,0,0);

  tx_str(TAGS,"rb");
  tx_str(TEXT,"radiobutton1");
  tx_06(RBUTTON,7,ROOT,0,60,60);
  tx_10(FONT,0,RBUTTON,7,1,12,1,0,0,0);

  tx_str(TEXT,"radiobutton2");
  tx_06(RBUTTON,8,ROOT,0,60,85);
  tx_10(FONT,0,RBUTTON,8,2,12,0,1,1,0);

  tx_str(TEXT,"checkboxbutton");
  tx_06(CBUTTON,9,ROOT,0,60,110);
  tx_10(FONT,0,CBUTTON,9,3,8,0,0,0,1);

  tx_02(PRINTOBJECTS,1);
}
```



```
#include <winard.h>

void setup()
{ Serial.begin(115200);
  build_up();
}

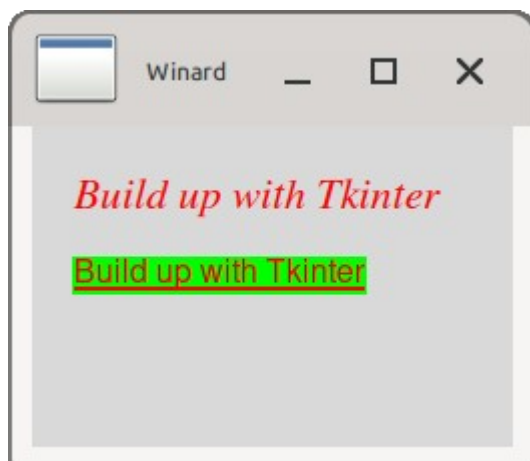
void loop() {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

  tx_str(TEXT,"Build up with Tkinter");
  tx_06(LABEL,3,ROOT,0,20,25);
  tx_10(FONT,0,LABEL,3,1,16,0,1,0,0);
  tx_04(CFG, RED); //color foreground
  tx_05(STYLE,0,LABEL,3,CFG);

  tx_06(LABEL,4,ROOT,0,20,65);
  tx_10(FONT,0,LABEL,4,0,12,0,0,1,0);
  tx_05(STYLE,0,LABEL,4,CFG);
  tx_04(CBG, GREEN); //color background
  tx_05(STYLE,0,LABEL,4,CBG);

  tx_02(PRINTOBJECTS,1);
}
```



The following statements are also valid:

```
tx_10(FONT,0,BUTTON,6,0,16,0,1,0,0);    only button 6
```

```
tx_10(FONT,0,THEMESTYLE,0,2,8,0,1,0,0);  for all widgets of the theme
```

```
tx_10(FONT,0,SET,0,2,10,0,1,0,0);
```

```
tx_05(THEMESTYLE,0,BUTTON,0,FONT);      all buttons of the theme
```

BUTTON is an example, also valid for RBUTTON , CBUTTON , etc.

Change the theme with:

```
tx_02(THEME,*);
```

where:

```
* = 1 ; clam
```

```
* = 2 ; alt
```

```
* = 3 ; default
```

```
* = 4 ; classic
```

4.3 Buttons style

4.3.1 button, radiobutton and checkbutton

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

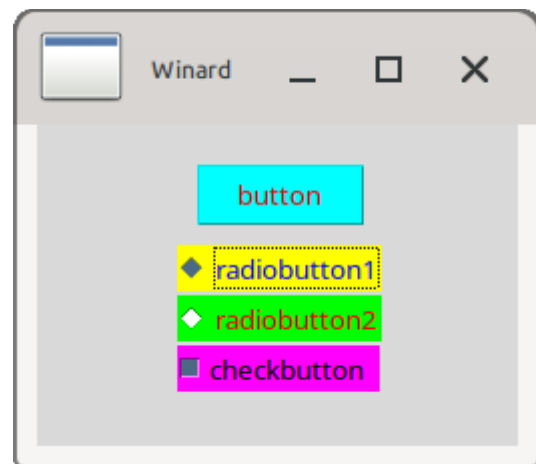
  tx_str(TEXT,"button");
  tx_06(BUTTON,16,ROOT,0,80,20);
  tx_04(CFG,255,0,0);
  tx_05(STYLE,0,BUTTON,16,CFG);
  tx_04(CBG,0,255,255);
  tx_05(STYLE,0,BUTTON,16,CBG);

  tx_str(TAGS,"rb");
  tx_str(TEXT,"radiobutton1");
  tx_06(RBUTTON,17,ROOT,0,70,60);
  tx_04(CFG,0,0,255);
  tx_05(STYLE,0,RBUTTON,17,CFG);
  tx_04(CBG,255,255,0);
  tx_05(STYLE,0,RBUTTON,17,CBG);

  tx_str(TEXT,"radiobutton2");
  tx_06(RBUTTON,18,ROOT,0,70,85);
  tx_04(CFG,255,0,255);
  tx_05(STYLE,0,RBUTTON,18,CFG);
  tx_04(CBG,0,255,0);
  tx_05(STYLE,0,RBUTTON,18,CBG);

  tx_str(TEXT,"checkboxbutton  ");
  tx_06(CBUTTON,19,ROOT,0,70,110);
  tx_04(CFG,0,255,0);
  tx_05(STYLE,0,CBUTTON,19,CFG);
  tx_04(CBG,255,0,255);
  tx_05(STYLE,0,CBUTTON,19,CBG);

  tx_02(PRINTOBJECTS,1);
}
```



4.3.2 Style map

Exept to change the foreground and background colors, it is het possible to change the selected and the not-selected fore- and backgrounds

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

  tx_str(TEXT,"button");
  tx_06(BUTTON,16,ROOT,0,80,20);
  tx_str(TAGS,"rb");
  tx_str(TEXT,"radiobutton1");
  tx_06(RBUTTON,17,ROOT,0,70,60);
  tx_str(TEXT,"radiobutton2");
  tx_06(RBUTTON,18,ROOT,0,70,85);
  tx_str(TEXT," checkboxbutton ");
  tx_06(CBUTTON,19,ROOT,0,70,110);

  tx_04(CFG, WHITE);
  tx_04(CBG, BLACK);
  tx_05(STYLE,0,BUTTON,16,CFG);
  tx_05(STYLE,0,BUTTON,16,CBG);

  tx_04(CACTIVE_FG,ORANGE);
  tx_04(CSELECT, RED);
  tx_04(CNSELECT, YELLOW);

  tx_05(MAP,0, BUTTON,16,CFG);
  tx_05(MAP,0,RBUTTON,17,CFG);
  tx_05(MAP,0,RBUTTON,18,CFG);
  tx_05(MAP,0,CBUTTON,19,CFG);

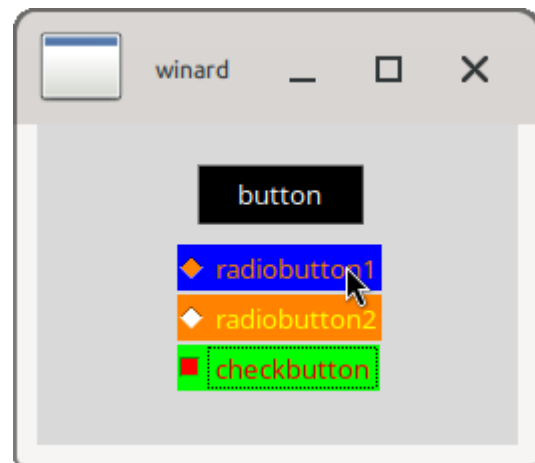
  tx_04(CACTIVE_FG, BLUE);
  tx_04(CSELECT, GREEN);
  tx_04(CNSELECT, ORANGE);

  tx_05(MAP,0, BUTTON,16,CBG);
  tx_05(MAP,0,RBUTTON,17,CBG);
  tx_05(MAP,0,RBUTTON,18,CBG);
  tx_05(MAP,0,CBUTTON,19,CBG);

  tx_04(CACTIVE_FG,MAGENTA);
  tx_04(CSELECT, RED);
  tx_04(CINDICATOR,WHITE);

  tx_05(MAP,0,RBUTTON,17,CIND);
  tx_05(MAP,0,RBUTTON,18,CIND);
  tx_05(MAP,0,CBUTTON,19,CIND);

  tx_02(PRINTOBJECTS,1);
}
```



4.3.3 Button size



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop( ) {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,320,240);

  tx_str(TEXT,"text");
  tx_06(BUTTON,3,ROOT,0,10,20);
  tx_06(BUTTON,4,ROOT,0,10,75);
  tx_06(BUTTON,5,ROOT,0,10,130);

  tx_06(BUTTON,13,ROOT,0,125,20);
  tx_06(BUTTON,14,ROOT,0,125,75);
  tx_06(BUTTON,15,ROOT,0,125,130);

  tx_06(BUTTON,23,ROOT,0,240,20);
  tx_06(BUTTON,24,ROOT,0,240,75);
  tx_06(BUTTON,25,ROOT,0,240,130);

  tx_02(BORDERWIDTH,3);
  tx_05(STYLE,0,BUTTON,4,BORDERWIDTH);
  tx_02(BORDERWIDTH,5);
  tx_05(STYLE,0,BUTTON,5,BORDERWIDTH);

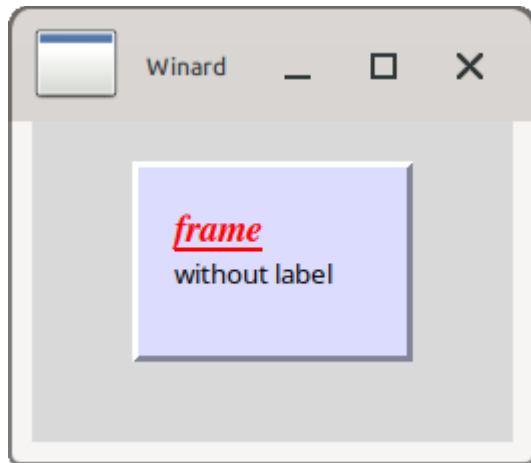
  tx_02(BORDERWIDTH,0);
  tx_05(STYLE,0,BUTTON,13,BORDERWIDTH);
  tx_02(BORDERWIDTH,3);
  tx_05(STYLE,0,BUTTON,14,BORDERWIDTH);
  tx_02(BORDERWIDTH,5);
  tx_05(STYLE,0,BUTTON,15,BORDERWIDTH);

  tx_02(WIDTH,5);
  tx_05(STYLE,0,BUTTON,14,WIDTH);
  tx_02(WIDTH,8);
  tx_05(STYLE,0,BUTTON,15,WIDTH);

  tx_02(WIDTH,2);
  tx_05(STYLE,0,BUTTON,23,WIDTH);
  tx_02(WIDTH,5);
  tx_05(STYLE,0,BUTTON,24,WIDTH);
  tx_02(WIDTH,8);
  tx_05(STYLE,0,BUTTON,25,WIDTH);

  tx_02(PRINTOBJECTS,1);
}
```

4.4 Frame



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

  tx_str(TEXT,"label");
  tx_02(POSITION,2);
  tx_08(LABELFRAME,1,ROOT,0,30,10,180,130);
  tx_04(CBG, 220,255,220);
  tx_05(STYLE,1,LABELFRAME,1,CBG);
  //      /--- 1 directs to the label
  tx_10(FONT,1,LABELFRAME,1,1,12,0,1,0,0);
  tx_02(RELIEF,2);
  // 0 flat, 1 groove, 2 raised,
  // 3 ridge, 4 solid, 5 sunken
  tx_05(STYLE,0,LABELFRAME,1,RELIEF);
  tx_02(BORDERWIDTH,3);
  tx_05(STYLE,0,LABELFRAME,1,BORDERWIDTH);

  tx_str(TEXT,"frame");
  tx_06(LABEL,4,LABELFRAME,1,40,25);
  tx_str(TEXT,"with label");
  tx_06(LABEL,5,LABELFRAME,1,40,45);
  tx_05(STYLE,0,LABELFRAME,1,CBG);
  tx_05(STYLE,0,LABEL,4,CBG);
  tx_05(STYLE,0,LABEL,5,CBG);

  tx_02(PRINTOBJECTS,1);
}
```

```
#include <winard.h>

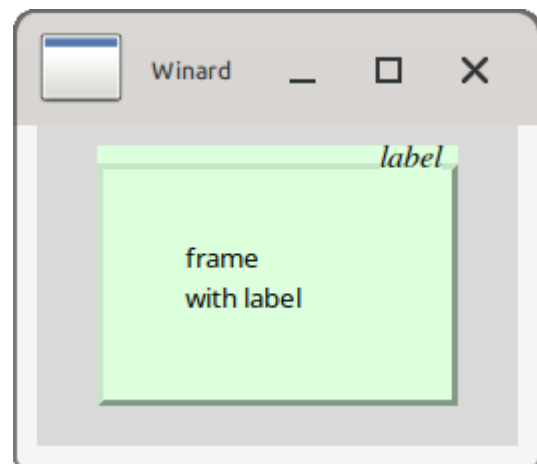
void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

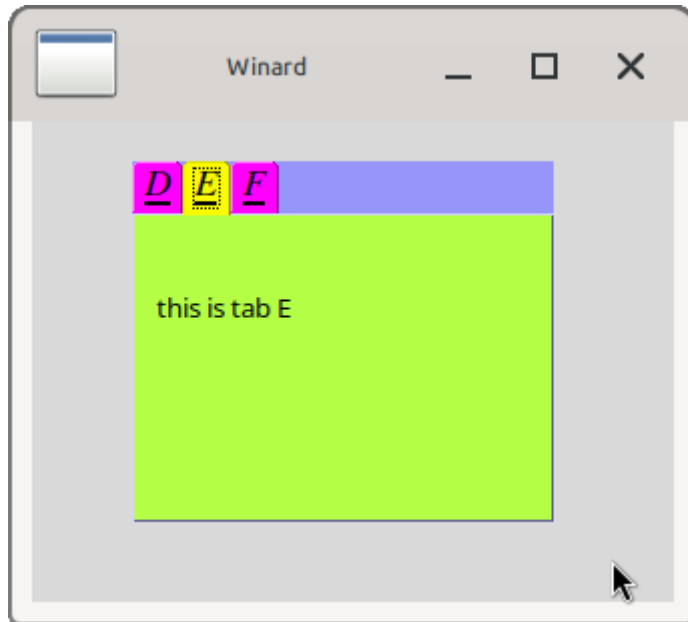
void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

  tx_08(FRAME,1,ROOT,0,50,20,140,100);
  tx_str(TEXT,"frame");
  tx_06(LABEL,4,FRAME,1,20,25);
  tx_04(CFG, 255,0,0);
  tx_05(STYLE,0,LABEL,4,CFG);
  tx_10(FONT,0,LABEL,4,1,14,1,1,1,0);
  tx_str(TEXT,"without label");
  tx_06(LABEL,5,FRAME,1,20,45);
  tx_04(CBG, 220,220,255);
  tx_05(STYLE,0,FRAME,1,CBG);
  tx_05(STYLE,0,LABEL,4,CBG);
  tx_05(STYLE,0,LABEL,5,CBG);
  tx_02(RELIEF,2);
  // 0 flat, 1 groove, 2 raised,
  // 3 ridge, 4 solid, 5 sunken
  tx_05(STYLE,0,FRAME,1,RELIEF);
  tx_02(BORDERWIDTH,3);
  tx_05(STYLE,0,FRAME,1,BORDERWIDTH);

  tx_02(PRINTOBJECTS,1);
}
```



4.5 Notebook



It is not possible to assign a custom numbering to the tabs. It starts with tab0, tab1, tab2 etc.

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,320,240);

  // the tabs start with tab 0,
  // the next, tab 1 , tab 2, etc.

  tx_str(TEXT,"D,E,F");
  tx_02(POSITION,0);
  tx_08(NOTEBOOK,2,ROOT,0,50,20,210,180);
  tx_str(TEXT,"this is tab D");
  tx_06(LABEL,14,TAB,0,10,35);
  tx_str(TEXT,"this is tab E");
  tx_06(LABEL,15,TAB,1,10,35);
  tx_str(TEXT,"this is tab F");
  tx_06(LABEL,16,TAB,2,10,35);

  tx_04(CBG,LIME);
  tx_05(STYLE,0,TAB,0,CBG);
  tx_05(STYLE,0,TAB,1,CBG);
  // style the label background too:
  tx_05(STYLE,0,LABEL,15,CBG);

  tx_04(CBG,MAGENTA);
  tx_05(STYLE,1,NOTEBOOK,2,CBG);
  //      \__ directs to tab

  tx_04(CBG,PURPLE);
  tx_05(STYLE,0,NOTEBOOK,2,CBG);
  //      \__ directs to notebook

  tx_04(CSELECT,250,250,0);
  tx_05(MAP,1,NOTEBOOK,2,CSELECT);
  //      \__ directs to tab
  // MAP is a STYLE way too

  tx_10(FONT,0,LABEL,16,1,14,1,0,0,0);
  tx_10(FONT,1,NOTEBOOK,2,1,14,0,1,1,0);
  //      \__ directs to tab
  // FONT is the STYLE way for fonts

  tx_str(TEXT,"ok");
  tx_06(BUTTON,6,TAB,2,40,120);
  tx_02(WIDTH,3); // number of characters
  tx_05(STYLE,1,BUTTON,6,WIDTH);
  tx_04(CBG,150,250,50);
  tx_05(STYLE,1,BUTTON,6,CBG);
  tx_02(PRINTOBJECTS,1);
}
```

4.6 Menubutton

The menu button is in ttk style.

The menu according to tk.

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  the_build();
}

void loop() {}

void the_build()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

  tx_04(CFG,255,0,0);
  tx_04(CBG,0,255,0);
  tx_04(CSELECT_FG,0,0,255);
  tx_04(CAFG,255,255,0);
  tx_04(CABG,0,255,255);
  tx_10(FONT,0,SET,0,1,12,0,0,1,0);
  tx_08(MENUOPTIONS,1,1,1,1,1,1,1);
  //          1 2 3 4 5 6 7
  // 1-tear 2-font 3-foreground 4-background
  // 5-activeforeground 6-activebackground
  // 7-selectcolor
  tx_str(TEXT,"select");
  tx_06(MBUTTON,2,ROOT,0,80,20);

  tx_04(CBG,220,220,255);
  tx_05(STYLE,0,MBUTTON,2,CBG);
  tx_04(CFG,200,0,0);
  tx_05(STYLE,0,MBUTTON,2,CFG);
  tx_10(FONT,0,MBUTTON,2,1,14,0,1,0,0);

  tx_str(TEXT,"check");
  tx_04(MCBUTTON,3,MBUTTON,2);
  tx_str(TAGS,"rb");
  tx_str(TEXT,"radio1");
  tx_04(MRBUTTON,5,MBUTTON,2);
  tx_str(TEXT,"radio2");
  tx_04(MRBUTTON,6,MBUTTON,2);
  tx_10(FONT,1,MRBUTTON,6,1,14,0,0,1,0);

  tx_02(PRINTOBJECTS,1);
}
```



4.7 Menubar

The menu bar widget is not a themed tk (ttk) functionality.

This means that no style options can be executed afterwards.

All configurations must be present before the widget is created.

This also applies to its components, first define the properties then the component.



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,290,220);

  tx_04(CFG,255,0,0);
  tx_04(CBG,0,255,0);
  tx_04(CSELECT_FG,0,0,255);
  tx_04(CAFG,255,255,0);
  tx_04(CABG,0,255,255);
  tx_10(FONT,0,SET,0,1,13,0,1,1,0);
  tx_08(MENUOPTIONS,1,1,1,1,1,1,1);
  // menubar options: 1 2 3 4 5 6 7
  // 1-tear 2-font 3-foreground 4-background
  // 5-activeforeground 6-activebackground
  // 7-selectcolor

  tx_04(CFG,0,255,0);
  tx_04(CBG,0,0,255);
  tx_04(CSELECT_FG,255,0,0);
  tx_04(CAFG,0,255,255);
  tx_04(CABG,255,0,255);
  tx_10(FONT,0,SET,0,1,11,0,0,0,0);
  tx_08(MENUOPTIONS2,1,1,1,1,1,1,1);
  // barmenu and submenu options

  tx_02(TEAR,1);
  tx_str(TEXT,"menu1,menu2,menu3");
  tx_04(MENUBAR,0,ROOT,0);

  tx_str(TEXT,"button1");
  tx_04(BMCOMMAND,1,BARMENU,0);
  tx_04(BMSEPARATOR,2,BARMENU,0);
  tx_str(TEXT,"submenu1");
  tx_04(CBG,255,0,255);
  tx_08(MENUOPTIONS2,1,1,1,1,1,1,1);
  tx_04(SUBMENU,0,BARMENU,0);
  tx_str(TEXT,"button2");
  tx_04(BMCOMMAND,2,SUBMENU,0);
  tx_str(TEXT,"button3");
  tx_04(BMCOMMAND,3,SUBMENU,0);
  tx_str(TEXT,"submenu2");
  tx_10(FONT,0,SET,0,1,11,0,1,0,0);
  tx_04(CFG,0,0,0);
  tx_04(CBG,255,255,0);
  tx_08(MENUOPTIONS2,1,1,1,1,1,1,1);
  tx_04(SUBMENU,2,SUBMENU,0);
  tx_str(TEXT,"button4");
  tx_04(BMCOMMAND,4,SUBMENU,2);

  tx_str(TAGS,"rb2");
  //TAGS, connects the radio buttons
  // with this label
  tx_str(TEXT,"radio1");
  tx_04(BMRADIO,1,BARMENU,0);
  tx_str(TEXT,"radio2");
  tx_04(CBG,0,255,255);
  tx_08(MENUOPTIONS2,1,1,1,1,1,1,1);
  tx_04(BMRADIO,2,BARMENU,0);
  tx_str(TEXT,"check1");
  tx_04(BMCHECK,1,BARMENU,0);

  tx_str(TEXT,"button5");
  tx_04(BMCOMMAND,5,BARMENU,1);
  tx_str(TEXT,"button6");
  tx_04(BMCOMMAND,6,BARMENU,1);

  tx_02(PRINTOBJECTS,1);
}
```

4.8 Slider or scale

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {;}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,180);

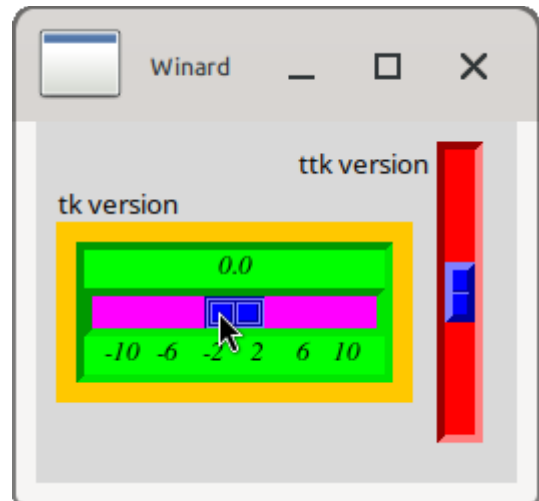
  tx_str(TEXT,"tk version");
  tx_06(LABEL,1,ROOT,0,10,30);

  tx_04(CBG,0,255,0);
  tx_04(CABG,0,0,255);
  tx_04(CTROUGH,255,0,255);
  tx_04(CHLBG,255,200,0); //highlightbackground
  tx_04(CHL,0,200,255);  //highlight
  tx_02(SLIDERWIDTH,16);
  tx_02(RELIEFWIDTH,4);
  tx_02(BORDERWIDTH,10);
  tx_02(RELIEFTROUGH,3);
  tx_02(RELIEFSLIDER,1);
  tx_10(FONT,0,SET,0,1,10,0,1,0,0);
  tx_12(SCALEOPTIONS,1,1,1,1,1,1,1,1,1, 1, 1);
  //      1 2 3 4 5 6 7 8 9 10 11
  //option use, 0 not 1 yes:
  // 1-font 2-background 3-activebackground
  // 4-troughcolor 5-highlightcolor
  // 6-highlightbackground 7-sliderwidth
  // 8-reliefwidth 9-borderwidth
  // 10-relief 11-sliderrelief
  tx_str(TEXT,"0 1 4.0 -10.0 10.0 2.0 1");
  tx_08(SCALE,18,ROOT,0,10,50,150,12);

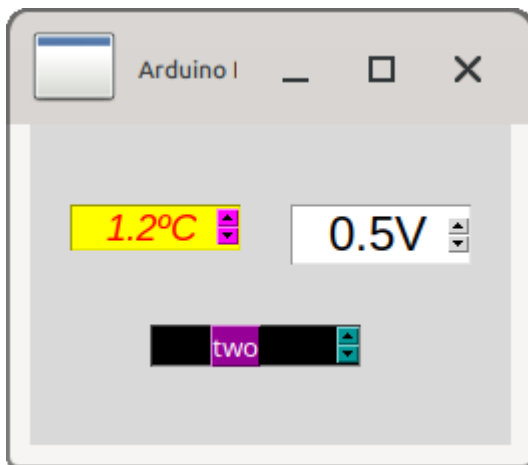
  tx_str(TEXT,"ttk version");
  tx_06(LABEL,1,ROOT,0,130,10);
  tx_str(TEXT,"1 1 2.0 20.0 -20.0 2.0 2");
  tx_08(SCALE,19,ROOT,0,200,10,0,150);

  tx_04(CSLIDER,0,0,255);
  tx_05(STYLE,0,SCALE,19,CSLIDER);
  tx_04(CTROUGH,255,0,0);
  tx_05(STYLE,0,SCALE,19,CTROUGH);
  tx_02(SLIDERWIDTH,15);
  tx_05(STYLE,0,SCALE,19,SLIDERWIDTH);
  tx_02(BORDERWIDTH,4);
  tx_05(STYLE,0,SCALE,19,BORDERWIDTH);
  tx_02(RELIEFTROUGH,1);
  tx_05(STYLE,0,SCALE,19,RELIEFTROUGH);

  tx_02(PRINTOBJECTS,1);
}
```



4.9 Spinbox



```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Arduino Display");
  tx_04(ROOT,0,240,160);

  tx_str(SPINBOXSET,"0 2 6 1.0°C");
  tx_str(TEXT,"0.0 10.0 0.1 %.1f°C ");
  tx_10(FONT,0,SPINBOX,1,2,14,0,1,0,0);
  tx_06(SPINBOX,1,ROOT,0,20,40);

  tx_04(CFG, 255,0,0);
  tx_04(CBG, 255,0,255);
  tx_04(CSELECT_FG,0,0,255);
  tx_04(CSELECT_BG,255,100,0);
  tx_04(CFIELD_BG, 255,255,0);
  tx_05(STYLE,0,SPINBOX,1,CFG);
  tx_05(STYLE,0,SPINBOX,1,CBG);
  tx_05(STYLE,0,SPINBOX,1,CFIELD_BG);
  tx_05(STYLE,0,SPINBOX,1,CSELECT_BG);
  tx_05(STYLE,0,SPINBOX,1,CSELECT_FG);

  // remember, next line not longer then the
  // buffer length, including TEXT, spaces
  // and the newline/feedback characters
  tx_str(TEXT,"-0.2V -0.1V 0.0V 0.1V 0.2V 0.5V 1.0V");
  tx_str(SPINBOXSET,"1 2 5 0.2V");
  tx_10(FONT,0,SPINBOX,2,2,18,0,0,0,0);
  tx_06(SPINBOX,2,ROOT,0,130,40);

  tx_str(SPINBOXSET,"1 1 10 four");
  tx_str(TEXT,"one two three four five");
  tx_06(SPINBOX,3,ROOT,0,60,100);

  tx_04(CFG, 255,200,255);
  tx_04(CBG, 0,150,150);
  tx_04(CFIELD_BG,0,0,0);
  tx_04(CSELECT_FG,255,255,255);
  tx_04(CSELECT_BG,150,0,150);
  tx_05(STYLE,0,SPINBOX,3,CFG);
  tx_05(STYLE,0,SPINBOX,3,CBG);
  tx_05(STYLE,0,SPINBOX,3,CFIELD_BG);
  tx_05(STYLE,0,SPINBOX,3,CSELECT_FG);
  tx_05(STYLE,0,SPINBOX,3,CSELECT_BG);
}
```

4.10 Entry

```
#include <winard.h>

void setup( )
{ Serial.begin(115200);
  build_up();
}

void loop() {};

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

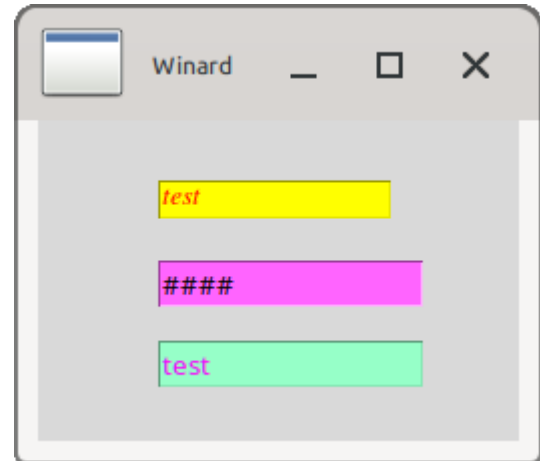
  tx_02(WIDTH,16);
  tx_10(FONT,0,ENTRY,3,1,10,0,1,0,0);
  tx_06(ENTRY,3,ROOT,0,60,30);

  tx_04(CFG, 255,0,0);
  tx_04(CFIELD_BG,255,255,0);
  tx_05(STYLE,0,ENTRY,3,CFG);
  tx_05(STYLE,0,ENTRY,3,CFIELD_BG);

  tx_str(SHOW,"#");
  tx_06(ENTRY,14,ROOT,0,60,70);
  tx_04(CFG, 0,0,0);
  tx_04(CFIELD_BG,255,100,255);
  tx_05(STYLE,0,ENTRY,14,CFG);
  tx_05(STYLE,0,ENTRY,14,CFIELD_BG);

  tx_str(SHOW,"");
  tx_06(ENTRY,23,ROOT,0,60,110);
  tx_04(CFG,255,0,255);
  tx_04(CFIELD_BG,150,255,200);
  tx_05(STYLE,0,ENTRY,23,CFG);
  tx_05(STYLE,0,ENTRY,23,CFIELD_BG);

  tx_02(PRINTOBJECTS,1);
}
```



5 Interaction PC-window and microcontroller

From now on, MC is used when referring to the microcontroller and PC when referring to the window launched by Winard.

Three situations can be distinguished in actions and signals:

1. From MC to PC

Two examples:

- 1.a one with a digital input signal, chapter 5.1

- 1.b one with an analog input signal, chapter 5.2

2. From PC to MC

Two examples:

- 2.a one with a digital output; switch leds on off, 5.3

- 2.b one with an analog output; led brightness with slider, pulse width modulation, 5.4

3. From PC to MC to PC

One example:

- 3.a draw functions; pc buttons actions, and display the microcontroller reaction, 5.5

Data transfer PC → MC

Transferring data from the MC to the PC has now been used extensively in all the examples of building a Winard window. Transferring data from a PC to the MC (MC->PC) hasn't yet been discussed.

The data is generated by buttons, sliders, spinboxes and entries. There are three types: integers, character strings and floats. By calling the `readcode()` function, the data is read from the MC input buffer. This data is stored in the structure:

```
struct bag
{ int i1=0, i2=0, i3=0, i4=0;
  char str[64];
  float fl=0.0;
} Bag;
```

Examples, the data of buttons, type 1:

1 25 6 0	or	1 BUTTON 6 0	push button
1 26 7 7	or	1 RBUTTON 7 7	radio button
1 27 9 0	or	1 CBUTTON 9 0	check button off
1 27 9 1	or	1 CBUTTON 9 1	check button on

There are more button types, used in menus, but we won't use them as examples here.

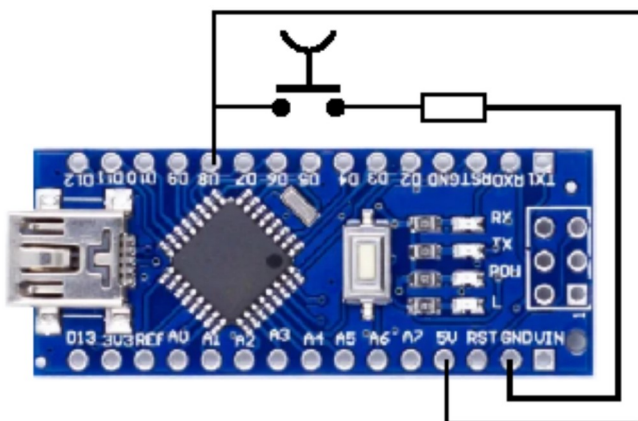
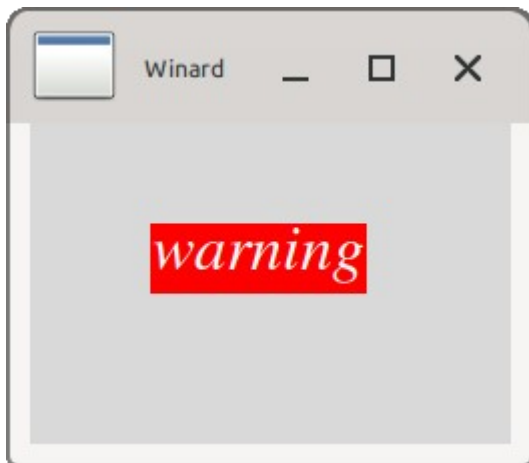
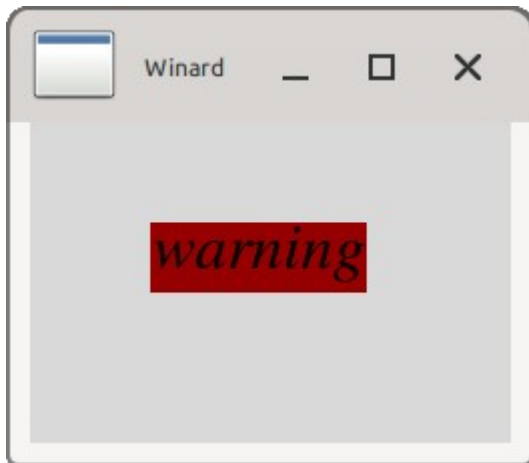
Then the content of Bag would be:

```
{ i1=1;      // data type
  i2=25;     // or 26 or 27
  i3=6;      // serial number of the button.
  i4=0;      // 0 or 1 check buttons, int. value if slider
  fl=0.0;    // only slider, float
  str[]="";  // filled by spinbox or entry
}
```

In general:

type	action	sn	value
			if type=1: 0 or 1 at a check button otherwise nothing
			if type=2: character string; spinbox or entry
			if type=3: int or float; slider
			serial number of the widget
			(push, radio or check)button, entry, spinbox or slider
			1=buttons, 2=spinbox or entry, 3=slider

5.1 Display an MC digital input signal on Winard



```
#include <winard.h>

const int buttonPin = 8;
int buttonState = 0;
int mem=0;

void setup()
{ Serial.begin(115200);
  build_up();
  pinMode(buttonPin, INPUT);
}

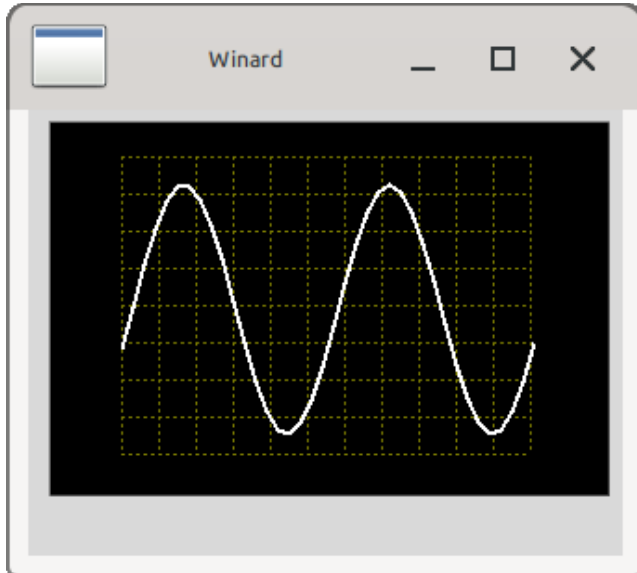
void loop()
{ buttonState = digitalRead(buttonPin);
  if ((buttonState == HIGH) & (mem==0))
  { tx_04(CBG, 255,0,0);
    tx_05(STYLE,0,LABEL,3,CBG);
    tx_04(CFG,255,255,255);
    tx_05(STYLE,0,LABEL,3,CFG);
    mem=1;
  }
  if ((buttonState == LOW) & (mem==1))
  { tx_04(CBG, 150,0,0);
    tx_05(STYLE,0,LABEL,3,CBG);
    tx_04(CFG,0,0,0);
    tx_05(STYLE,0,LABEL,3,CFG);
    mem=0;
  }
}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,160);

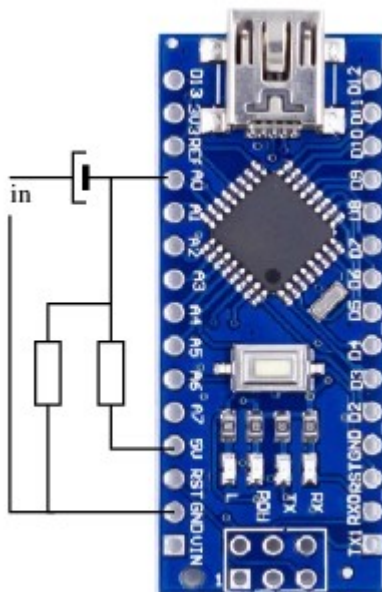
  tx_str(TEXT,"warning");
  tx_06(LABEL,3,ROOT,0,60,50);
  tx_10(FONT,0,LABEL,3,1,24,0,1,0,0);
  tx_04(CFG,0,0,0);
  tx_05(STYLE,0,LABEL,3,CFG);
  tx_04(CBG, 150,0,0);
  tx_05(STYLE,0,LABEL,3,CBG);

  tx_02(PRINTOBJECTS,1);
}
```

5.2 Draw an MC analog input signal on Winard



This signal has a frequency of 48 Hz.



```
#include <winard.h>

const int input = A0;
int buf[64];

void setup( )
{ Serial.begin(115200);
  build_up();
  // analogReference(INTERNAL);
  // analogReference(EXTERNAL);
  analogReference(DEFAULT);
  Serial.setTimeout(50);
}

void loop()
{ func();
}

void build_up()
{
  tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,1,320,240);

  tx_04(CFG,255,255,255);
  tx_04(CBG,0,0,0);
  tx_08(CANVAS,1,ROOT,0,10,5,300,200);

  tx_05(NLINES,9,12,20,20);
  tx_04(CFG,120,120,0);
  tx_02(LINEWIDTH,1);
  tx_04(DASH,1,2,2);
  tx_08(VLINES,3,CANVAS,1,40,20,40,180);
  tx_08(HLINES,3,CANVAS,1,40,20,260,20);
  tx_04(CFG,255,255,255);
  tx_04(DASH,0,8,8);

  tx_04(SETLINE,1,40,6);
  //      |   |   |_fix.distances X/Y dir.
  //      |   |   |__startvalue X or Y
  //      |   |   |__=0 line with X,Y amplitudes
  //      |   |   |__=1 line, only Y amplitudes
  //      |   |   |__=2 line, only X amplitudes
  tx_str(TAGS,"two");
  tx_02(LINEWIDTH,2);

  tx_02(PRINTOBJECTS,1);
  tx_02(PRINTBUILD,0);
}

void func()
{ int a=0;
  while (a<(38))
  { buf[a++]=analogRead(A0);delay(1);
  }
  a=0;
  tx_02(COORD,buf[a++]/6+15);
  while (a<(38))
  { tx_02(COORDAPPEND,buf[a++]/6+15);delay(2);
  }
  tx_str(TAGSDEL,"two");
  tx_04(NEWLINE,9,CANVAS,1);
}
```

5.3 Switch LEDs connected to the MC on or off via Winard.

```
#include <winard.h>

int red=5, green=6, blue=7;

void setup( )
{ Serial.begin(115200);
  build_up();
  Serial.setTimeout(200);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
}

void loop( )
{ readcode();
  if (Bag.i2==CBUTTON)
  { if (Bag.i3==1)
    { if (Bag.i4==1) digitalWrite(red, HIGH);
      if (Bag.i4==0) digitalWrite(red, LOW);
    }
    if (Bag.i3==2)
    { if (Bag.i4==1) digitalWrite(green,HIGH);
      if (Bag.i4==0) digitalWrite(green,LOW);
    }
    if (Bag.i3==3)
    { if (Bag.i4==1) digitalWrite(blue, HIGH);
      if (Bag.i4==0) digitalWrite(blue, LOW);
    }
  }
}

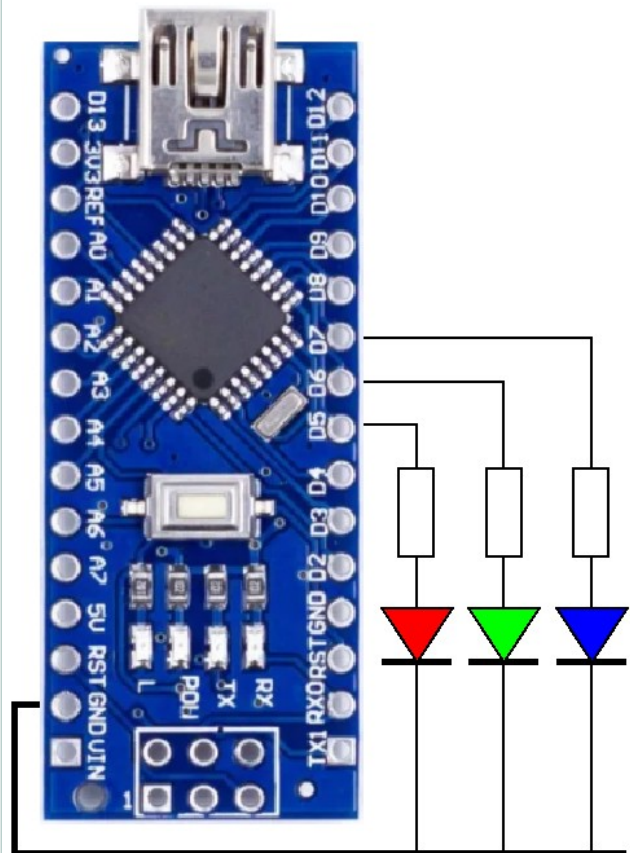
void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,40);

  tx_str(TEXT,"red  ");
  tx_06(CBUTTON,1,ROOT,0,10,10);
  tx_04(CFG,0,0,0);
  tx_04(CBG,255,0,0);
  tx_05(STYLE,0,CBUTTON,1,CFG);
  tx_05(STYLE,0,CBUTTON,1,CBG);

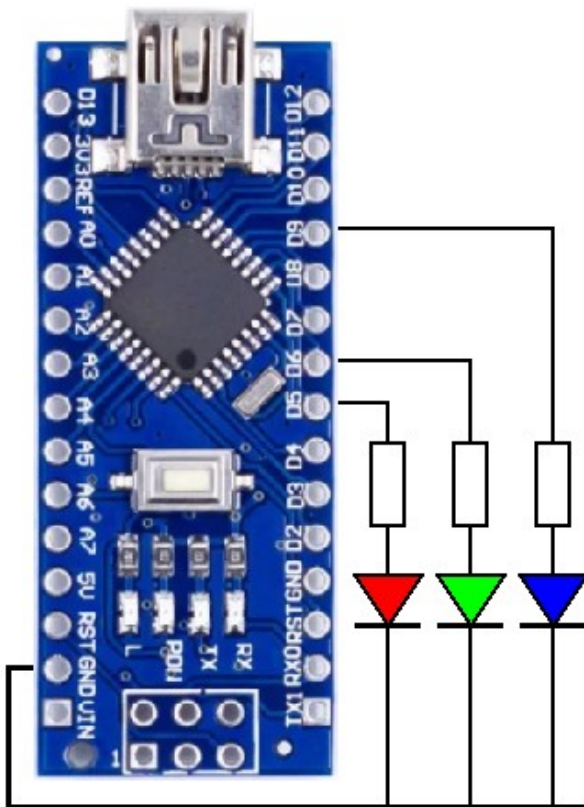
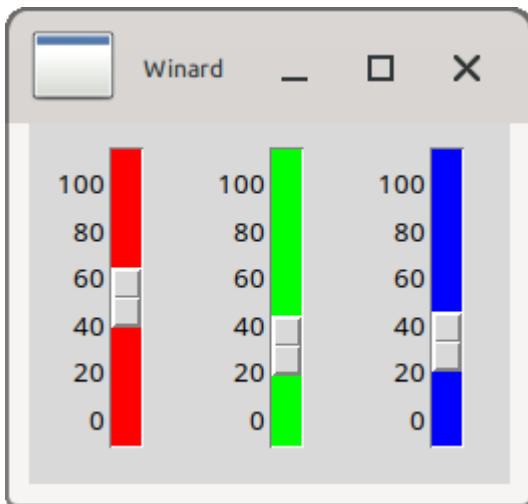
  tx_str(TEXT,"green");
  tx_06(CBUTTON,2,ROOT,0,95,10);
  tx_04(CBG,0,255,0);
  tx_05(STYLE,0,CBUTTON,2,CFG);
  tx_05(STYLE,0,CBUTTON,2,CBG);

  tx_str(TEXT,"blue ");
  tx_06(CBUTTON,3,ROOT,0,180,10);
  tx_04(CFG,255,255,255);
  tx_04(CBG,0,0,255);
  tx_05(STYLE,0,CBUTTON,3,CFG);
  tx_05(STYLE,0,CBUTTON,3,CBG);

  tx_02(PRINTOBJECTS,1);
  // tx_02(PRINTBUILD,0);
}
```



5.4 Led brightness with slider



```
// arduino nano

#include <winard.h>

const int red  =5, green=6, blue =9;

void setup( )
{ Serial.begin(115200);
  build_up();
  Serial.setTimeout(200); // reaction speed
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(9, OUTPUT);
}

void loop( )
{ readcode();
  if (Bag.i2==SLIDER)
  { switch(Bag.i3)
    { case 1: analogWrite(red,  2*Bag.i4); break;
      case 2: analogWrite(green,2*Bag.i4); break;
      case 3: analogWrite(blue, 2*Bag.i4); break;
    }
  }
}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,0,240,180);

  tx_str(TEXT,"0 0 20 100 0 0 0");
  tx_04(CTROUGH,255,0,0);
  tx_12(SCALEOPTIONS,0,0,0,1,0,0,0,0,0,0,0);
  tx_08(SCALE,1,ROOT,0,10,10,15,150);

  tx_str(TEXT,"0 0 20 100 0 0 0");
  tx_04(CTROUGH,0,255,0);
  tx_12(SCALEOPTIONS,0,0,0,1,0,0,0,0,0,0,0);
  tx_08(SCALE,2,ROOT,0,90,10,15,150);

  tx_str(TEXT,"0 0 20 100 0 0 0");
  tx_04(CTROUGH,0,0,255);
  tx_12(SCALEOPTIONS,0,0,0,1,0,0,0,0,0,0,0);
  tx_08(SCALE,3,ROOT,0,170,10,15,150);

  tx_02(PRINTOBJECTS,1);
}
```

5.5 Draw functions

```
#include <winard.h>
int buf[64];

void setup( )
{ Serial.begin(115200);
  Serial.setTimeout(50);
  build_up();
}

void loop()
{ readcode();
  if (Bag.i2==26)
  { switch(Bag.i3)
    { case 1: tx_04(CFG,255,0,0);break;
      case 2: tx_04(CFG,255,255,0);break;
      case 3: tx_04(CFG,0,255,0);break;
      case 4: tx_04(CFG,0,255,255);break;
      case 5: tx_04(CFG,0,0,255);break;
      case 6: tx_04(CFG,255,0,255);break;
      case 7: tx_04(CFG,255,255,255);break;
    }
  }
  if (Bag.i2==25)
  { switch(Bag.i3)
    { case 1: tx_str(TAGS,"func1"); funcsin(); break;
      case 2: tx_str(TAGS,"func2"); funcsquare(); break;
      case 3: tx_str(TAGS,"func3"); functriangle(); break;
      case 4: tx_str(TAGSDEL,"func1"); ; break;
      case 5: tx_str(TAGSDEL,"func2"); ; break;
      case 6: tx_str(TAGSDEL,"func3"); ; break;
    }
  }
}

void funcsin()
{ int a=0;
  tx_04(SETLINE,1,10,6);
  tx_02(COORD,(int) (50.0*sin(PI * a++/15)+70.0));
  while (a<31)
  {tx_02(COORDAPPEND,(int) (50.0*sin(PI * a++/15)+70.0));
  }
  tx_04(NEWLINE,7,CANVAS,1);
}

void funcsquare()
{ tx_04(SETLINE,0,10,6);
  tx_11(COORD,10,100,10,40,55,40,55,100,100,100);
  tx_11(COORDAPPEND,100,40,145,40,145,100,190,100,190,40);
  tx_04(NEWLINE,8,CANVAS,1);
}

void functriangle()
{ tx_04(SETLINE,1,10,45);
  tx_06(COORD,100,40,100,40,100);
  tx_04(NEWLINE,9,CANVAS,1);
}

void build_up()
{ tx_02(PRINTBUILD,1);
  tx_str(TEXT,"Winard");
  tx_04(ROOT,1,320,240);

  tx_02(WIDTH,8);
  tx_05(THEMESTYLE,0,RBUTTON,0,WIDTH);
```

```

tx_str(TAGS,"rb");
tx_str(TEXT,"red      ");
tx_06(RBUTTON,1,ROOT,0,230,20);
tx_04(CBG,255,0,0);
tx_05(STYLE,0,RBUTTON,1,CBG);
tx_str(TEXT,"yellow   ");
tx_06(RBUTTON,2,ROOT,0,230,50);
tx_04(CBG,255,255,0);
tx_05(STYLE,0,RBUTTON,2,CBG);
tx_str(TEXT,"green");
tx_06(RBUTTON,3,ROOT,0,230,80);
tx_04(CBG,0,255,0);
tx_05(STYLE,0,RBUTTON,3,CBG);
tx_str(TEXT,"cyan");
tx_06(RBUTTON,4,ROOT,0,230,110);
tx_04(CBG,0,255,255);
tx_05(STYLE,0,RBUTTON,4,CFG);
tx_05(STYLE,0,RBUTTON,4,CBG);
tx_str(TEXT,"blue  ");
tx_06(RBUTTON,5,ROOT,0,230,140);
tx_04(CFG,255,255,255);
tx_04(CBG,0,0,255);
tx_05(STYLE,0,RBUTTON,5,CFG);
tx_05(STYLE,0,RBUTTON,5,CBG);
tx_str(TEXT,"magenta");
tx_06(RBUTTON,6,ROOT,0,230,170);
tx_04(CBG,255,0,255);
tx_05(STYLE,0,RBUTTON,6,CBG);
tx_str(TEXT,"white");
tx_06(RBUTTON,7,ROOT,0,230,200);
tx_04(CBG,255,255,255);
tx_05(STYLE,0,RBUTTON,7,CBG);

tx_str(TEXT,"sinus");
tx_06(BUTTON,1,ROOT,0,10,160);
tx_str(TEXT,"square");
tx_06(BUTTON,2,ROOT,0,80,160);
tx_str(TEXT,"triangle");
tx_06(BUTTON,3,ROOT,0,150,160);
tx_str(TEXT,"delete");
tx_06(BUTTON,4,ROOT,0,10,190);
tx_str(TEXT,"delete");
tx_06(BUTTON,5,ROOT,0,80,190);
tx_str(TEXT,"delete");
tx_06(BUTTON,6,ROOT,0,150,190);

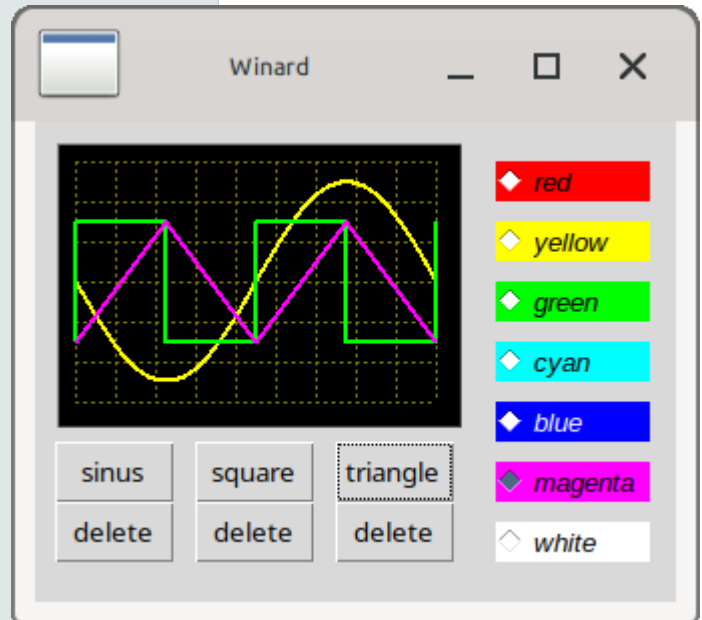
tx_02(WIDTH,6);
tx_05(THEMESTYLE,0,BUTTON,0,WIDTH);

tx_10(FONT,0,SET,0,2,10,0,1,0,0);
tx_05(THEMESTYLE,0,RBUTTON,0,FONT);

tx_04(CFG,255,255,255);
tx_04(CBG,0,0,0);
tx_08(CANVAS,1,ROOT,0,10,10,200,140);
tx_05(NLINES,7,10,20,20);
tx_04(CFG,120,120,0);
tx_02(LINEWIDTH,1);
tx_04(DASH,1,2,2);
tx_08(VLINES,3,CANVAS,1,10,10,10,130);
tx_08(HLINES,3,CANVAS,1,10,10,190,10);
tx_04(CFG,255,255,255);
tx_04(DASH,0,8,8);
tx_02(LINEWIDTH,2);

tx_02(PRINTOBJECTS,1);
tx_02(PRINTBUILD,0);
}

```



6 Winard.h

```
# define SET 1
# define TEXT 2
# define TAGS 3
# define SHOW 4
# define SPINBOXSET 5
# define TAGSDEL 6
# define COORD 8
# define COORDAPPEND 9
# define LINEAPPEND 9
# define COORDREPLACE 10
# define LINEREPLACE 10
# define NEWLINE 11
# define SETLINE 13
# define LINE 14
# define HLINES 15
# define V_LINES 16
# define RECT 17
# define OVAL 18
# define ARC 19
# define LABEL 24
# define BUTTON 25
# define RBUTTON 26
# define CBUTTON 27
# define SCALE 28
# define SLIDER 28
# define SPINBOX 29
# define ENTRY 30
# define LISTBOX 31
# define FRAME 36
# define LABELFRAME 37
# define CANVAS 38
# define NOTEBOOK 39
# define TAB 40 //?????
# define ROOT 41
# define TOPLEVEL 42
# define SEPARATOR 43
# define CANVASSELECT 44
# define MBUTTON 48
# define MCBUTTON 49
# define MRBUTTON 50
# define MENUBAR 51
# define BARMENU 52
# define BMCOMMAND 53
# define BMRADIO 54
# define BMCHECK 55
# define BMSEPARATOR 56
# define SUBMENU 57
# define ROOTCONFIG 58
# define INDON 59
# define TEAR 60
# define MENUOPTIONS 61
# define MENUOPTIONS2 62
# define SCALEOPTIONS 64
# define LINEWIDTH 65
# define DASH 66
# define ARROW 67
# define ARROWS 68
# define N_LINES 69
# define ARC_SHAPE 70 //
# define RELIEF1 71 // flat, groove, raised, ridge, solid, sunken
# define RELIEFTROUGH 72
# define RELIEF2 73
# define RELIEFSLIDER 73
# define FONT 80
# define POSITION 82
# define WIDTH 83
# define WIDTH1 83
# define SLIDERWIDTH 83
# define WIDTH2 84
# define BORDERWIDTH 84
# define WIDTH3 85
# define RELIEFWIDTH 85
# define COLOR1 88 // foreground CFG
# define CFG 88 // Style Map foreground color1
# define COLOR2 89 // background CBG CSLIDER
# define CBG 89 // Style Map background color2
# define CSLIDER 89 // slider color2
# define COLOR3 90 // CACTIVE_FG
# define CAFG 90 // active color3
```

```

# define CACTIVE_FG      90    //      Map
# define COLOR4          91    // CACTIVE_BG
# define CABG            91    // active          color4
# define CACTIVE_BG     91    //      Map
# define COLOR5          92    // CSELECT      CINDICATOR
# define CPFG            92    // press
# define CSELECT_FG     92    // select, color7, color5
# define CSELECT        92    //      Map selected color5
# define CINDICATOR     92
# define CIND           92
# define COLOR6          93    // CNSELECT
# define CPBG           93    // press          color6
# define CSBG           93    // select         color8
# define CSELECT_BG     93
# define CNSELECT       93    //      Map !selected color6
# define COLOR7          94
# define COLOR8          95    // CLIGHT
# define CLIGHT         95    // 27-04-2025
# define COLOR9          96    // CDARK
# define CNSFG          96    // press          color9
# define CDARK           96    // 27-04-2025
# define COLOR10         97
# define CFIELD_BG      97
# define COLOR11        98    indicator
# define COLOR12        99
# define CSELECTED      99
# define COLOR13        100
# define CHL            100    // highlightcolor
# define COLOR14        101
# define CHLBG          101    // highlightbackground
# define COLOR15        102
# define COLOR16        103    // troughcolor CTROUGH
# define CTROUGH        103    // scale troughcolor
# define PRINTOBJECTS   104
# define PRINTBUILD     105
# define STYLE          106
# define THEMESTYLE     107
# define MAP            108
# define THEME          109
# define MBOXINFO       112
# define MBOXWARNING    113
# define MBOXERROR      114
# define MBOXQUESTION   115
# define MBOXOKCANCEL   116
# define MBOXYESNO      117
# define MBOXRETRYCANCEL 118
# define RED            255,0,0
# define GREEN          0,255,0
# define BLUE           0,0,255
# define YELLOW         255,255,0
# define CYAN           0,255,255
# define MAGENTA        255,0,255
# define ORANGE         255,130,0
# define LIME           180,255,70
# define BLACK          0,0,0
# define WHITE          255,255,255
# define PURPLE         150,150,250

# define DELAY          1

char print_str[64];

void tx_02(int a,int b)
{ sprintf(print_str,"%d,%d",a,b);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_03(int a,int b,int c)
{ sprintf(print_str,"%d,%d,%d",a,b,c);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_04(int a,int b,int c,int d)
{ sprintf(print_str,"%d,%d,%d,%d",a,b,c,d);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_05(int a,int b,int c,int d,int e)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d",a,b,c,d,e);

```

```

    Serial.println(print_str);
    // delay(DELAY);
}

void tx_06(int a,int b,int c,int d, int e,int f)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,",a,b,c,d,e,f);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_07(int a,int b,int c,int d,int e, int f,int g)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_08(int a,int b,int c,int d,int e,int f, int g,int h)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g,h);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_09(int a,int b,int c,int d,int e,int f, int g,int h,int i)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g,h,i);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_10(int a,int b,int c,int d,int e,int f,int g, int h,int i,int j)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g,h,i,j);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_11(int a,int b,int c,int d,int e,int f,int g,int h, int i,int j,int k)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g,h,i,j,k);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_12(int a,int b,int c,int d,int e,int f,int g,int h, int i,int j,int k,int l)
{ sprintf(print_str,"%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,", a,b,c,d,e,f,g,h,i,j,k,l);
  Serial.println(print_str);
  // delay(DELAY);
}

void tx_str(int a,const char *str)
{ sprintf(print_str,"%d,%s", a,str);
  Serial.println(print_str);
  // delay(DELAY);
}

String str1;
char str2[64];

struct bag
{ int i1=0, i2=0, i3=0, i4=0;
  float fl;
  char str[64];
} Bag;

void readcode()
{ int a=0, len;
  Bag.i1=0, Bag.i2=0, Bag.i3=0, Bag.i4=0;
  if (Serial.available());
  { tx_str(0,"Start");
    str1=Serial.readString();
    for (a=0; a<64; a++) str2[a]=str1[a];
    sscanf(str2,"%d", &Bag.i1);
    switch(Bag.i1)
    { case 1: sscanf(str2,"%d%d%d%d",&Bag.i1,&Bag.i2,&Bag.i3,&Bag.i4);break;
      case 2: sscanf(str2,"%d%d%d%s",&Bag.i1,&Bag.i2,&Bag.i3,&Bag.str);break;
      case 3: sscanf(str2,"%d%d%d%s",&Bag.i1,&Bag.i2,&Bag.i3,&Bag.str);
              Bag.fl=atof(Bag.str);
              Bag.i4=Bag.fl; break;
    }
  }
  // tx_str(TEXT,Bag.str);
}
}

```

